



Data Pipeline II

Summer term 2025



About this class

Your instructor



Dr. Jannic Alexander Cutura

- › Jannic Cutura is an economist turned data scientist turned data engineer turned software engineer and currently works as a senior data engineer in the ECB's supervisory technologies and data team
- › Before, he worked in financial stability and monetary policy as data analyst/engineer. He also held previous appointments at the International Monetary Fund, the Bank for International Settlements. He holds an M.Sc. & Ph.D. in quantitative economics from Goethe university as well as a masters degree in Data Engineering from DSTI.
- › All views expressed in this lecture are solely his own and do not represent the views of the European Central Bank (ECB) or the Euro System of Central Bank (ESCB).
- › Second time teaching this class
- › He loves cheesecake 😊

Who are you?

Let's get to know each other

- › What's your name?
 - › Where do you live?
 - › What is your academic background? Social science, STEM?
 - › What is your professional background?
 - › What is your professional goal for post-master / the next 1-2 years?
 - › Favorite dessert?
-

This class focuses on the engineering aspect of data engineering. By the end will you build an end-to-end project, using IaC & CI/CD

- › Core curriculum for Data Engineering students at DSTI
- › This class focusses on data **engineering**
- › Most companies don't need "data science notebook hackers". They need:
 - › (data) engineers that can connect existing databases/datasets and build new ones
 - › AI engineers that can bring models to production and maintain an Mlflow logic
 - › Business Analysts that can derive actionable insights
- › Main learnings of this class:
 - › Common Data Engineering patterns
 - › Infrastructure as Code (IaC)
 - › Continuous Development / Continuous Implementation (CI/CD) pipelines
 - › ... And how to put those three together
 - › Know the (entire?) landscape, master some of it!

The class schedule

- › 27/06, 1400 – 1715: Intro + (advanced) data modelling
- › 02/07, 1400 – 1715: Code Design Patterns for Data Engineering
- › 21/07, 1400 – 1715: Unit Testing Spark
- › 28/07, 1400 – 1715: Advanced Spark techniques & Iceberg Table Format
- › 29/07, 1400 – 1715: Advanced Spark techniques & Iceberg Table Format
- › 30/07, 1400 – 1715: Terraform & CICD
- › 31/07, 1400 – 1715: Putting it all together (AWS Glue, terraform, CI/CD, airflow,...)
- › 01/08, 1400 – 1715: Putting it all together (AWS Glue, terraform, CI/CD, airflow,...)

A couple of setup notes to get you started....

- › All the code is on gitlab (<https://gitlab.com/dsti-data-pipeline-2>)
- › Install python 3.11+ on your system (<https://www.python.org/downloads/windows/>)
- › Install terraform on your system (<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>)
- › Install Docker (<https://docs.docker.com/desktop/setup/install/windows-install/>)
- › Install AWS CLI on your system (<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>)
- › Optional: Get an AWS Account. Secure it properly: <https://repost.aws/knowledge-center/security-best-practices>

Curriculum change log

› Fall 2024 → Summer 2025:

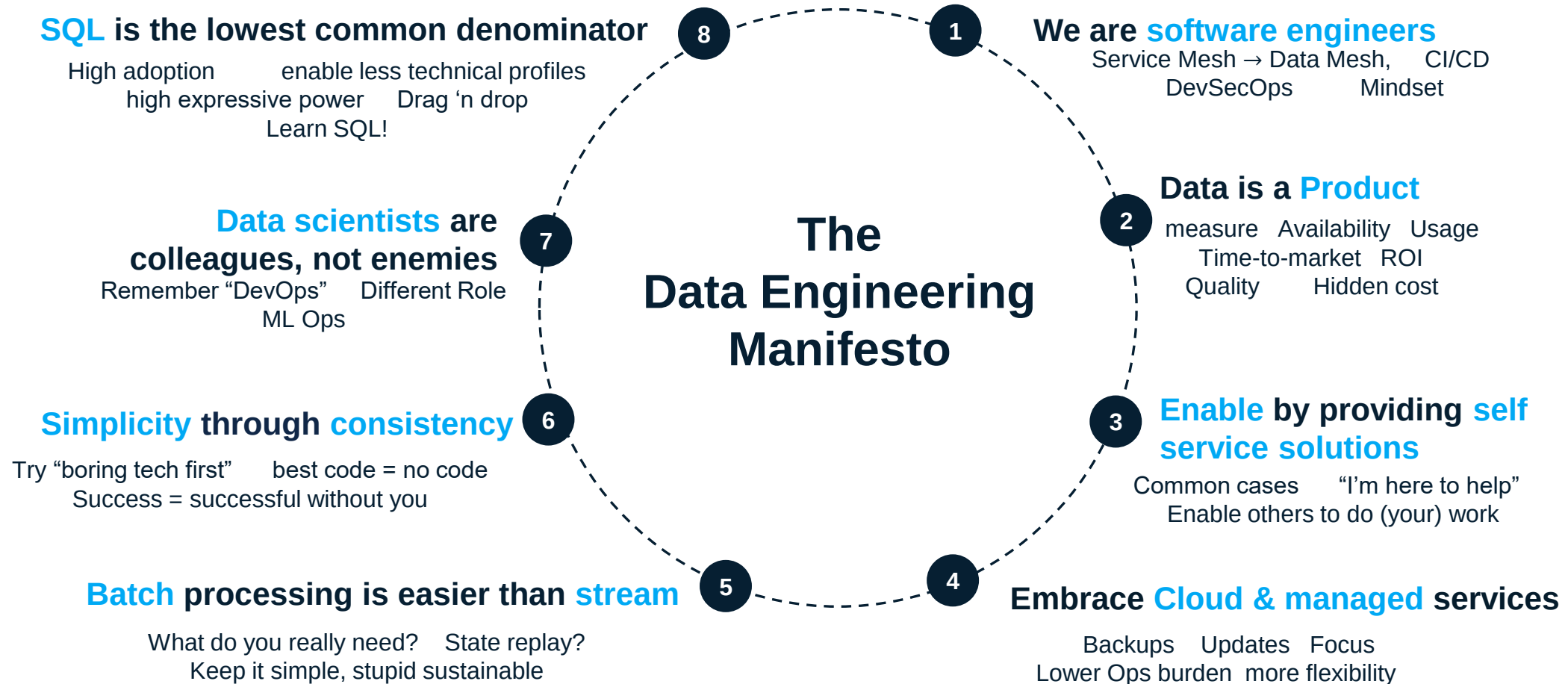
- › Added no-cloud terraform problem set (so students without AWS account can participate),
- › Added locally executable iceberg demo / problem sets (using Docker), so students with AWS Glue can work with iceberg
- › Shortened section on stream processing
- › Changed grading structure

› Summer 2025 → Fall 2025:

Overview

Know your data engineering landscape

Who is the data engineer? The Data Engineering Manifesto



Source: <https://www.dataminded.com/about>

The Data Engineering Canvas

Stakeholders



Business Units: Departments or teams that rely on data for decision-making.

Data Scientists & Analysts: Professionals who require clean, structured, and accessible data for analysis.

IT & Operations: Teams responsible for the infrastructure, deployment, and maintenance of data systems.

External Partners: Third-party data providers or consumers of the data.

Compliance & Legal: Ensuring the data engineering processes comply with legal and regulatory standards.

Data Infrastructure



Storage: Data lakes, databases, data warehouses, data lakehouse

Compute: Spark clusters, Kubernetes clusters, functions as a services

Orchestrators: Airflow, step functions

Cloud Providers: AWS, Google Cloud, Azure for scalable infrastructure.

Technologies & Tools



Data Management Platforms: Tools for managing data throughout its lifecycle.

Data Visualization: Tools like Tableau, Power BI, Looker for presenting data insights.

Data Versioning: Tools and methods for versioning data sets (e.g., DVC, LakeFS), modern table formats like Iceberg

Security Tools: Tools for encryption, identity management, and data masking.

Value Propositions



Data Accessibility: Ensuring that data is easily accessible and usable by the business or end-users.

Data Quality: Providing high-quality, accurate, and consistent data. This is typically ensure with subject experts from the business area

Scalability: Designing systems that can grow with increasing data volumes and user demands.

Real-time Data Processing: Enabling real-time or near-real-time data processing capabilities.

Cost Efficiency: Optimizing the cost of data storage, processing, and retrieval.

Data Security & Compliance: Ensuring data is secure and meets regulatory requirements.

Key Metrics



Data Latency: The time taken for data to move from source to destination.

Data Quality Metrics: Completeness, accuracy, consistency, timeliness, and validity of data.

Pipeline Reliability & Scalability: Uptime, failure rates, and time to recovery. Ability to handle increased data volume and user load.

Cost (per Terabyte): The cost of storing and processing data

Pipelines and Workflows



Ingestion: Methods for collecting and bringing data into the system.

Transformation: Processes to clean, aggregate, and transform data.

Orchestration & Monitoring: Scheduling and managing workflows (e.g., Apache Airflow).

Skills



Data Engineering Skills: Expertise in SQL, ETL, data warehousing, big data technologies.

Infrastructure as Code: Knowledge of how to deploy Cloud resources using IaC tools like terraform to create production grade applications.

Programming Skills: Proficiency in Python, Java, Scala, etc.

Cloud & DevOps Skills: Knowledge of cloud platforms, CI/CD pipelines, containerization.

Domain Knowledge: Understanding of the business domain to ensure relevant data engineering solutions.

Collaboration & Communication: Ability to work with cross-functional teams. Be humble, assume everyone knows something that you don't. Show up & work hard!

Cost & Resources



Infrastructure Costs: Expenses related to cloud storage, compute resources, and data tools.

Development Costs: Time and resources needed to develop and maintain data pipelines.

Personnel Costs: Salaries and training for data engineers, analysts, and other team members.

Licensing & Subscriptions: Costs associated with software, tools, and platforms.

Outcome & Deliverables



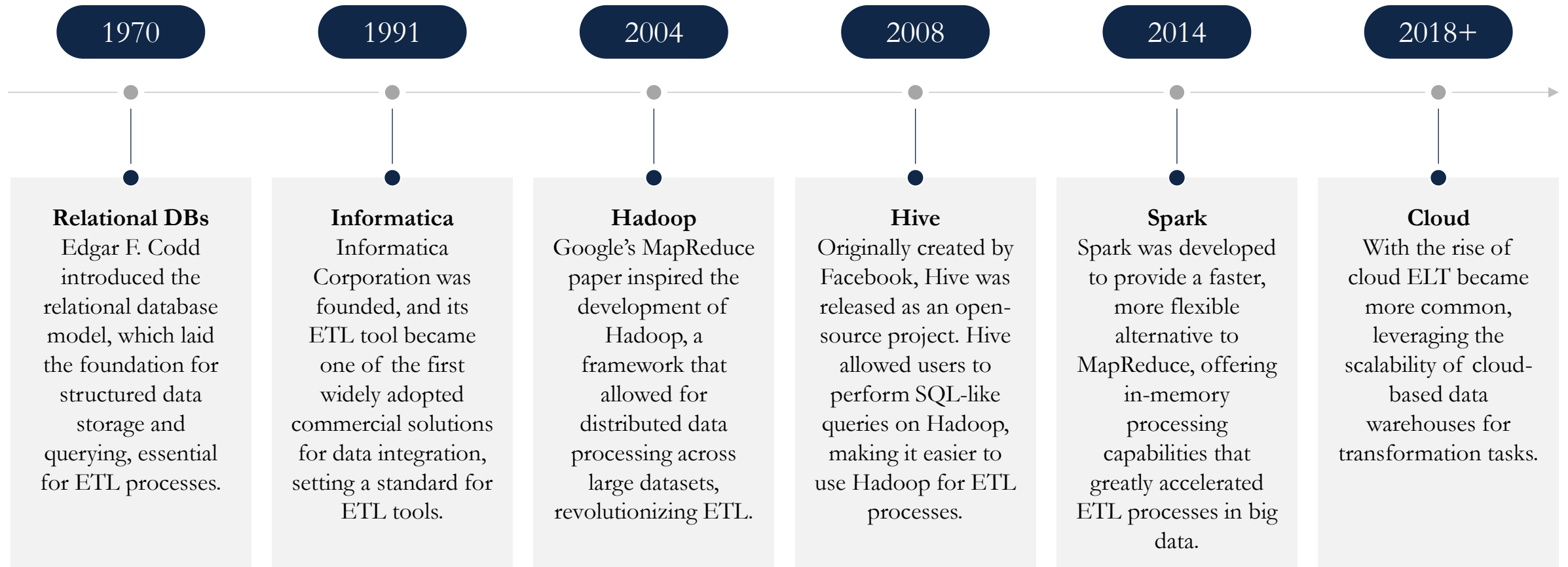
Data Products: Well-organized, accessible, and reliable data sets.

Dashboards & Reports: Visual and actionable insights for stakeholders.

Automation: Automated data workflows reducing manual intervention.

→ Improved Decision-Making: Enhanced ability to make data-driven decisions.

Evolution of Data Engineering

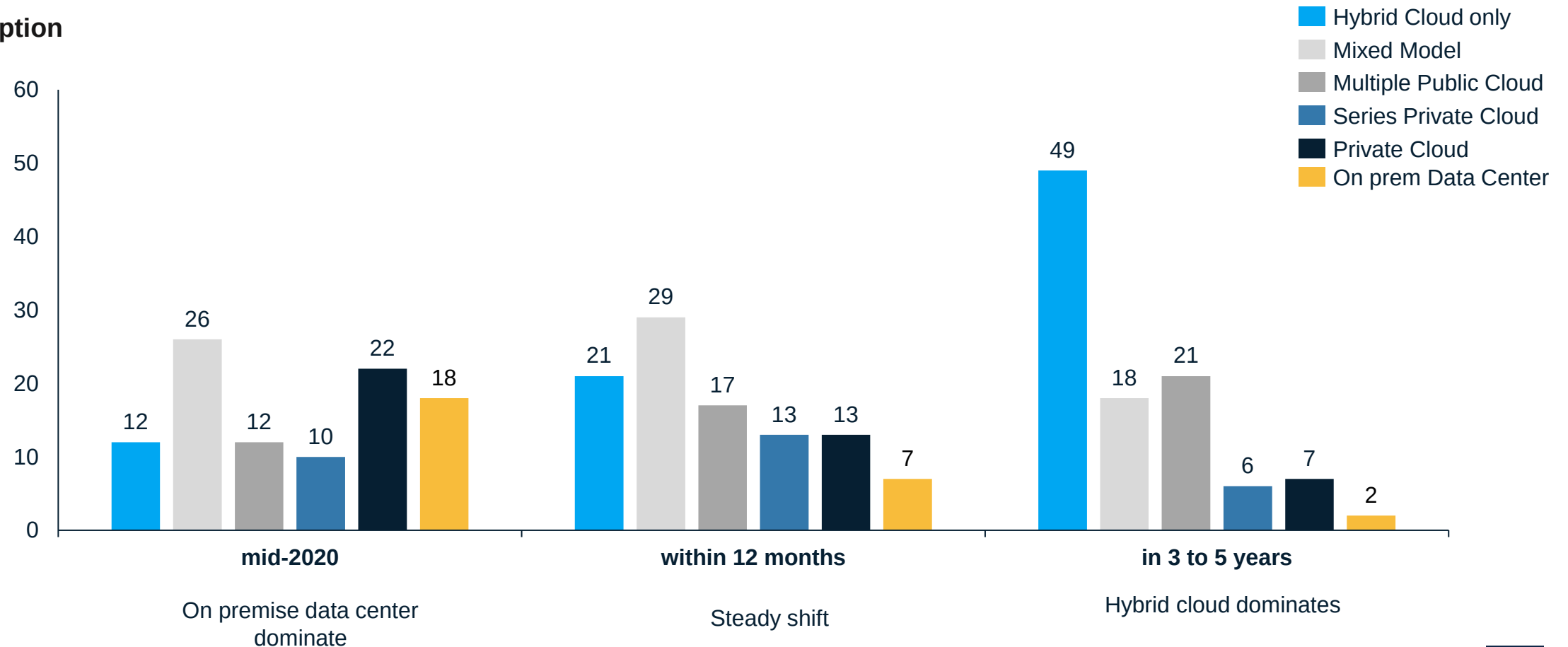


The trend is towards hybrid cloud solutions

Traditional Database/Datawarehouse systems

Cloud Adaption

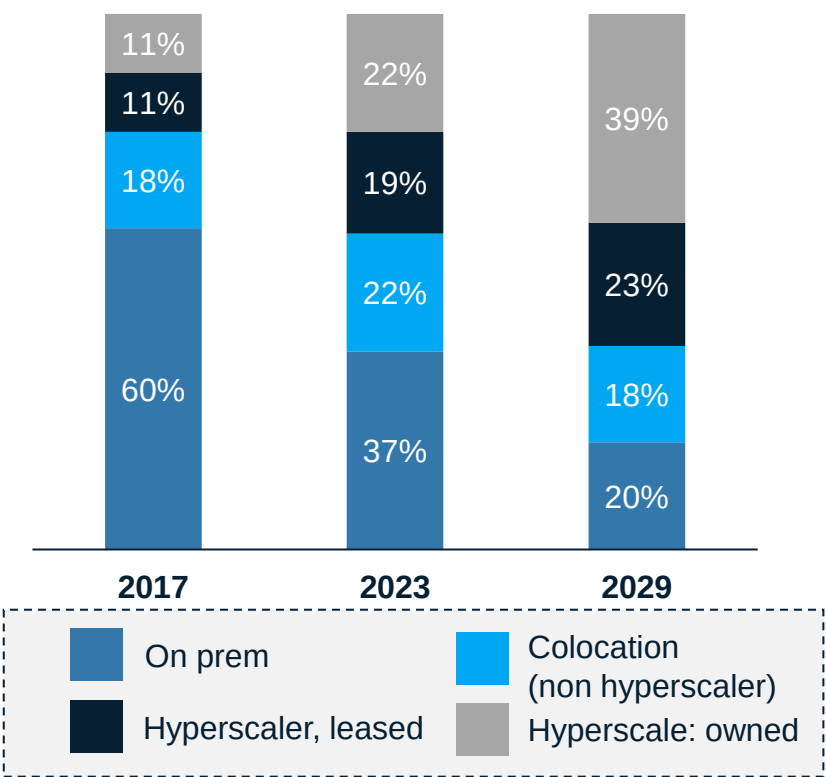
%



Data center capacity is dominated by hyperscalers (AWS, Azure, GCP) in the US

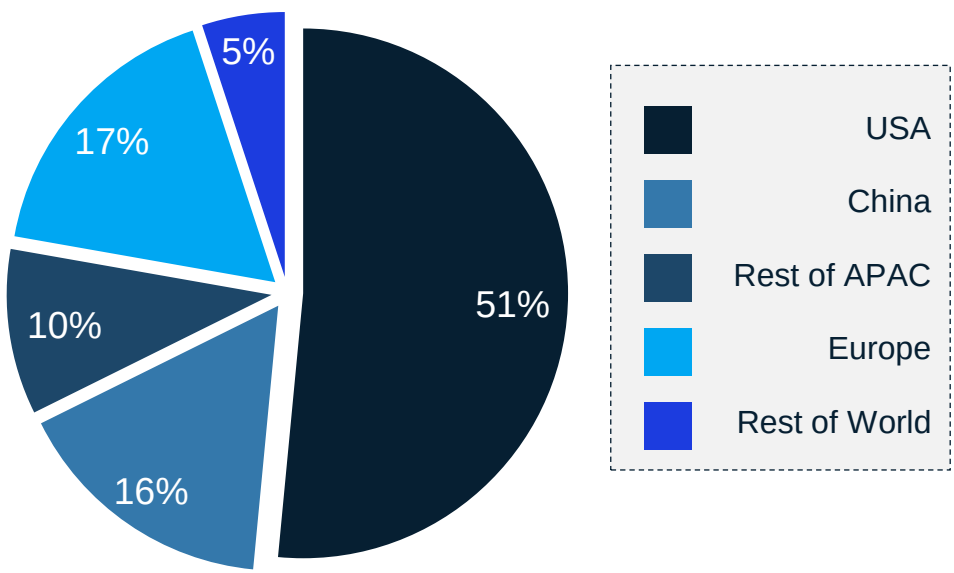
Data Center Capacity Trends

Share of Critical IT Load Worldwide, Percentage Points



USA dominate data center capacity

Data center capacity, Percentage Points



Source: <https://www.srgresearch.com/articles/hyperscale-operators-and-colocation-continue-to-drive-huge-changes-in-data-center-capacity-trends>

The cloud market is global, except China

AWS, Azure and GCP dominate, except for China, where Chinese cloud providers are in the lead

Rank	Worldwide	USA	China	Rest of APAC	Europe	Rest of world
1						
2			 Tencent Cloud			
3			 中国电信 CHINA TELECOM			
4		 ORACLE CLOUD	 HUAWEI	 NTT	 salesforce	 ORACLE CLOUD
5	 ORACLE CLOUD	 salesforce		 ORACLE CLOUD	 ORACLE CLOUD	 salesforce
6	 salesforce	 IBM Cloud		 salesforce	 IBM Cloud	 IBM Cloud

AWS, Azure, GCP dominate the market

The next 20 companies make up 26% of the market

The winners

Amazon Web Services (AWS)

Amazon's AWS is the largest cloud provider, offering extensive services and global reach.

Microsoft Azure

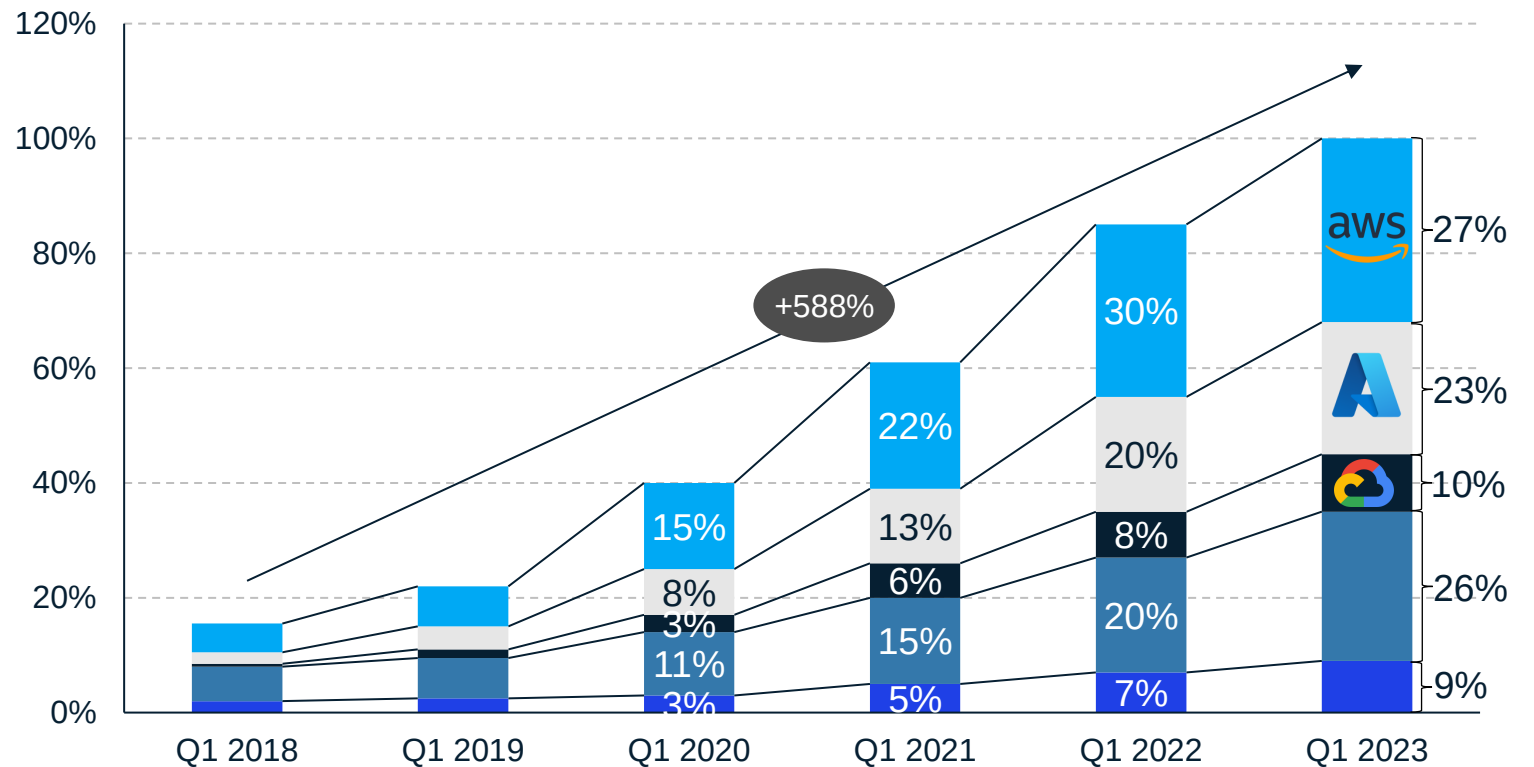
Microsoft's Azure provides diverse cloud services with strong integration into Microsoft products.

Google Public Cloud (GCP)

Google Cloud Platform excels in data analytics, machine learning, and open-source tools.

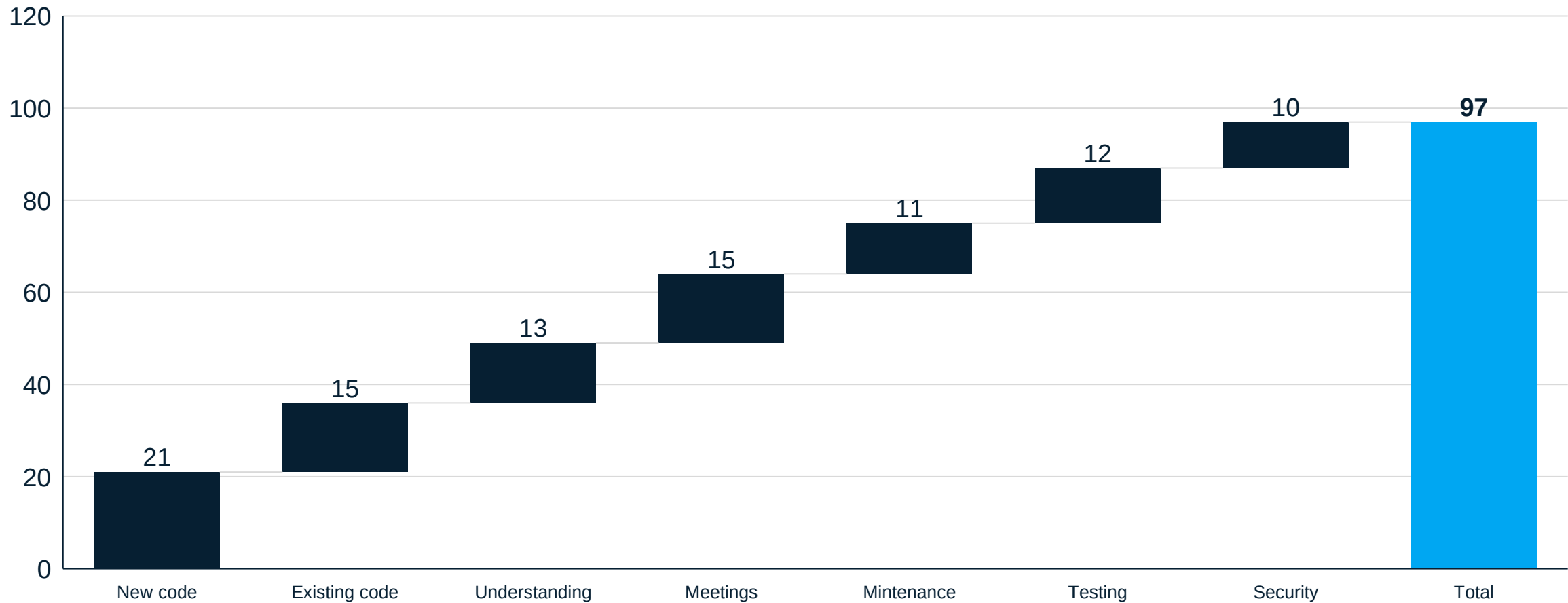


Worldwide Revenue
(100% = 63,7\$ billion USD)



Code generation use case had the highest adoption rate, while survey findings suggest the benefits of AI lie across the entire developer workflow.

Time developer spend on, Percentage Points

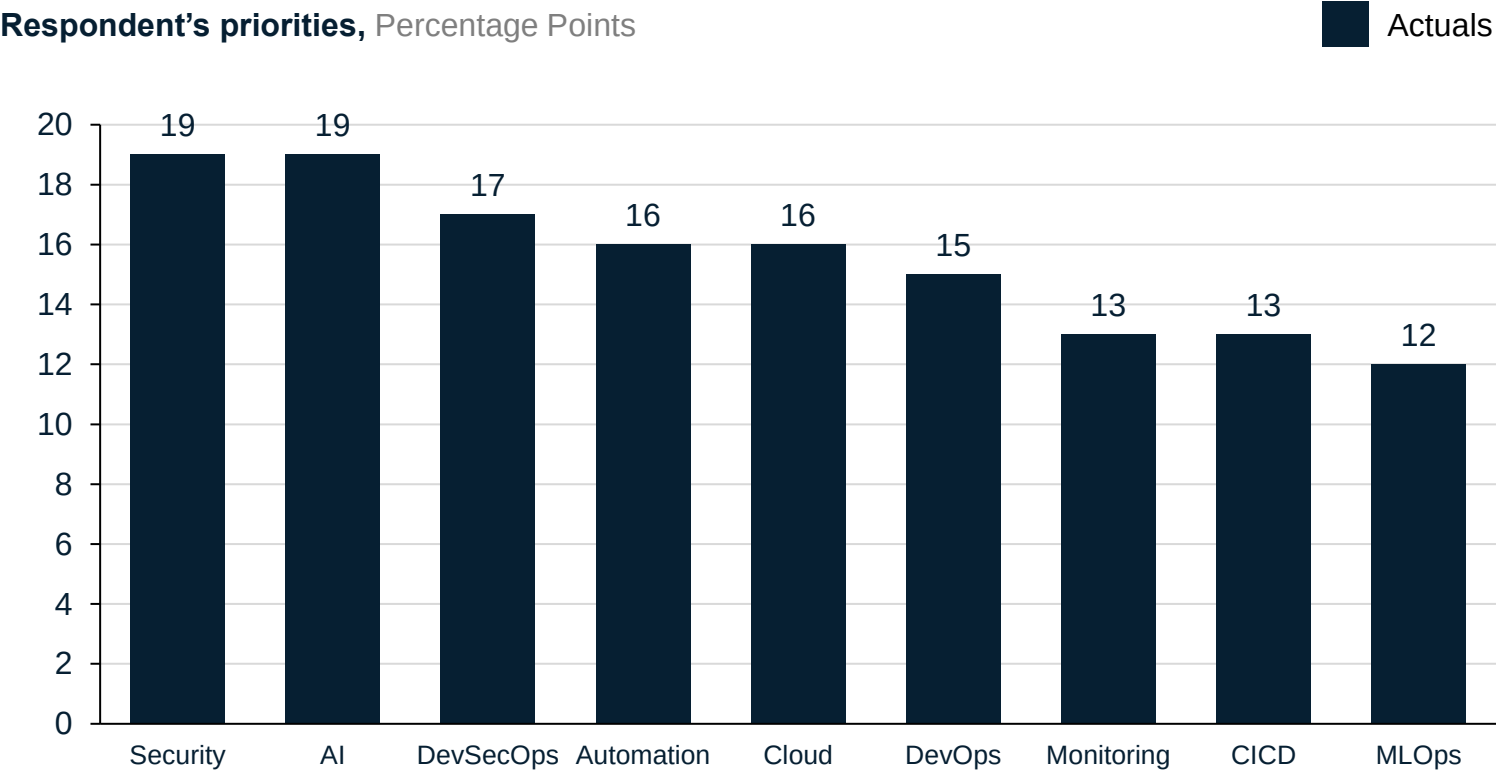


Source: Gitlab 2024 DevSecOps Report

Security remains a top priority despite a slight decrease in the percentage of respondents who identified security as a priority.

Top IT investment priorities in 2024

Reported below is the level of respondents reply



Highly regulated industries are doubling down on security

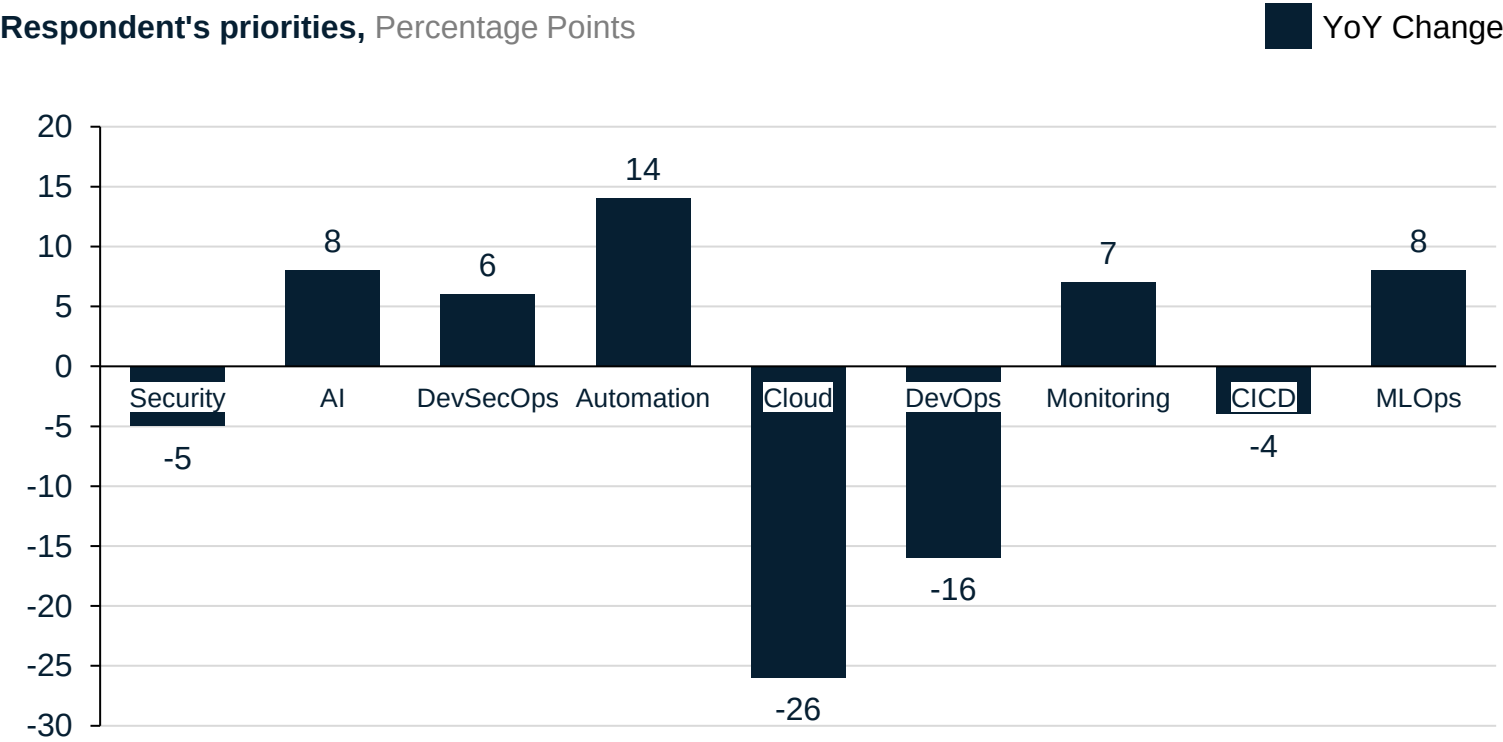
- The number of respondents who identified a DevSecOps platform as an investment priority and the number of respondents who said their organizations are currently using a DevSecOps platform this year were similar to 2023, suggesting that interest and investment in DevSecOps platforms remains stable. Meanwhile, interest in automation saw a significant increase in 2024, with automation jumping from priority number six in 2023 to number four this year.
- One of the biggest signals of the reshuffling of priorities is the fate of cloud computing: The cloud was the top priority in 2023, but this year it dropped to number five.

Footnote: Survey question, What are the top IT investment priorities for your organization in 2024?

AI showed a slight but statistically significant increase from 2023 — suggesting that AI is increasingly becoming a primary focus for many organizations.

Top IT investment priorities in 2024

Reported below is the change of developer priorities



Tech leads the way on AI

- However, it's clear that the cloud continues to be important — the survey shows a significant decrease year over year in the number of respondents who said they are running less than half of their apps in the cloud (68% in 2023 to 43% in 2024), and a corresponding increase in the number of respondents who said they are running 50% or more of their apps in the cloud (32% in 2023 to 55% in 2024).
- Finally, while metrics and analytics didn't make the top five investment priorities this year, we did observe a significant uptick compared to last year (9% in 2023 to 12% in 2024), suggesting that this is an up-and-coming priority for organizations.

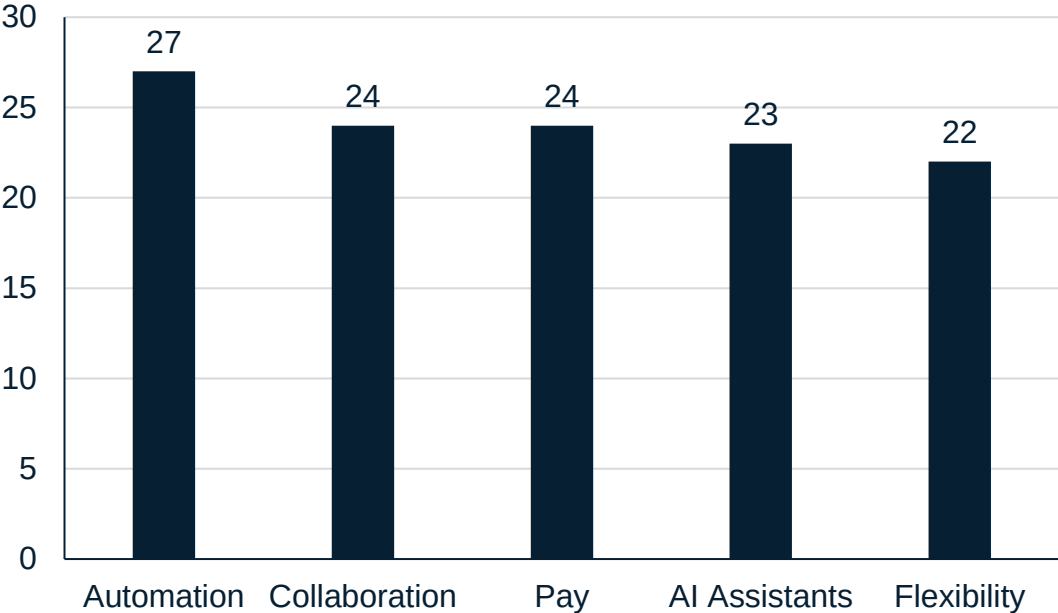
Footnote: Survey question, What are the top IT investment priorities for your organization in 2024?

What developers want: Automation, collaboration, and AI

67% of respondents said their software development lifecycle is mostly or completely automated.

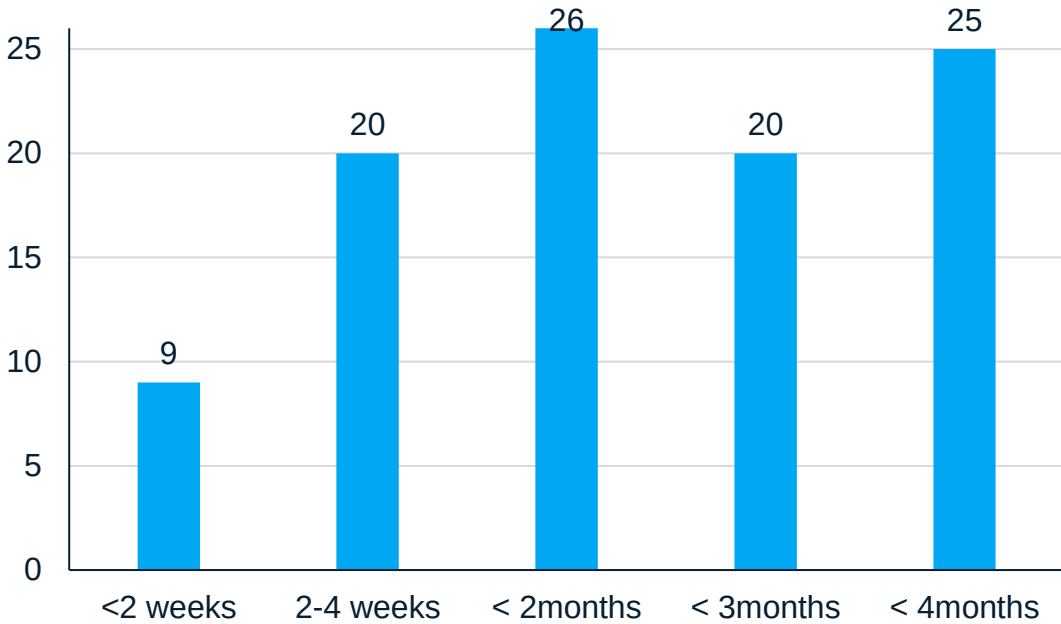
Top changes that could be made to improve developer satisfaction, according to developers

Respondents, Percentage Points



How long does it take to onboard new developers in your organization?

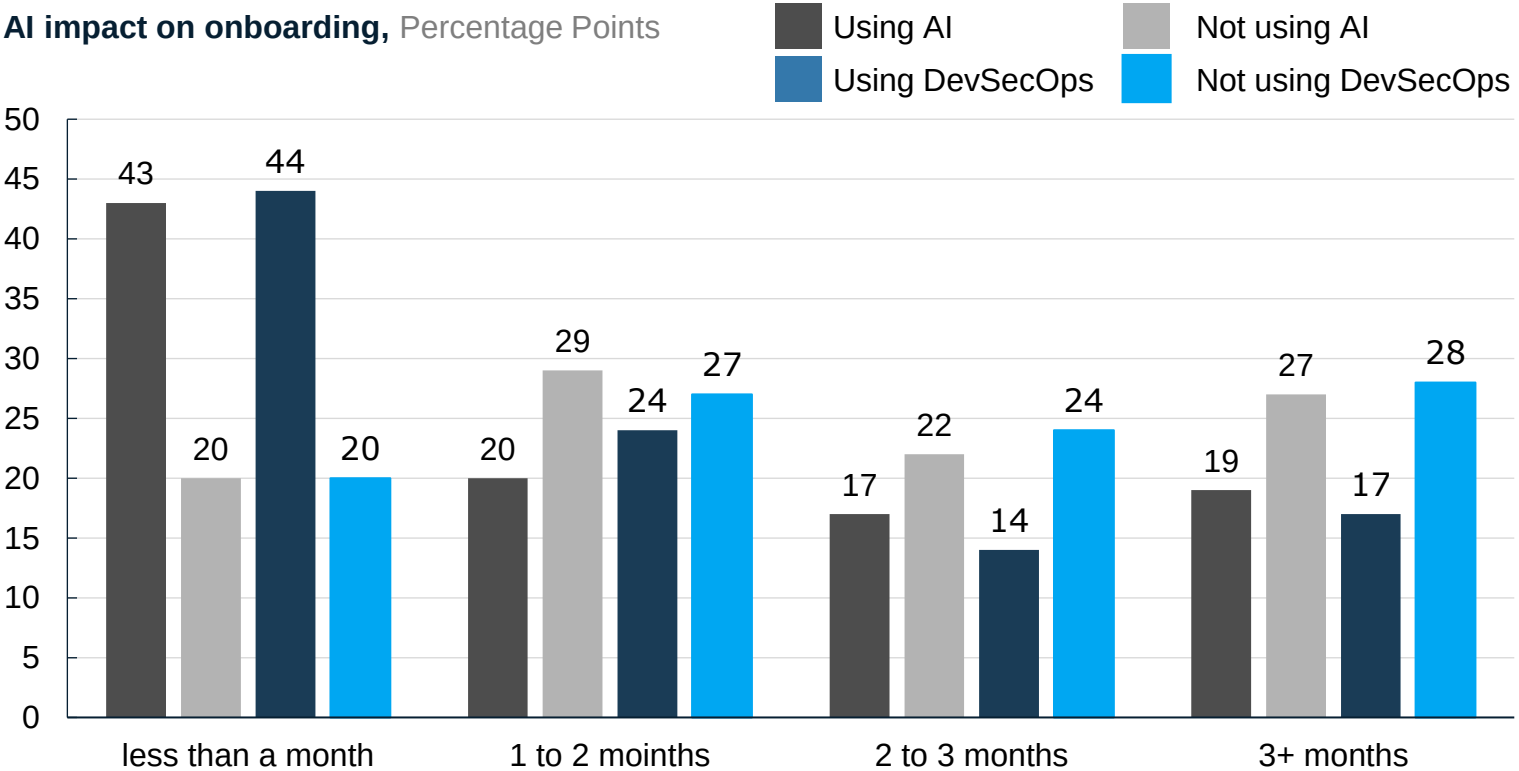
Onboarding time, time



AI and DevSecOps platform usage speed up developer onboarding

Companies ought to adopt AI and DevSecOps platforms to speed up onboarding

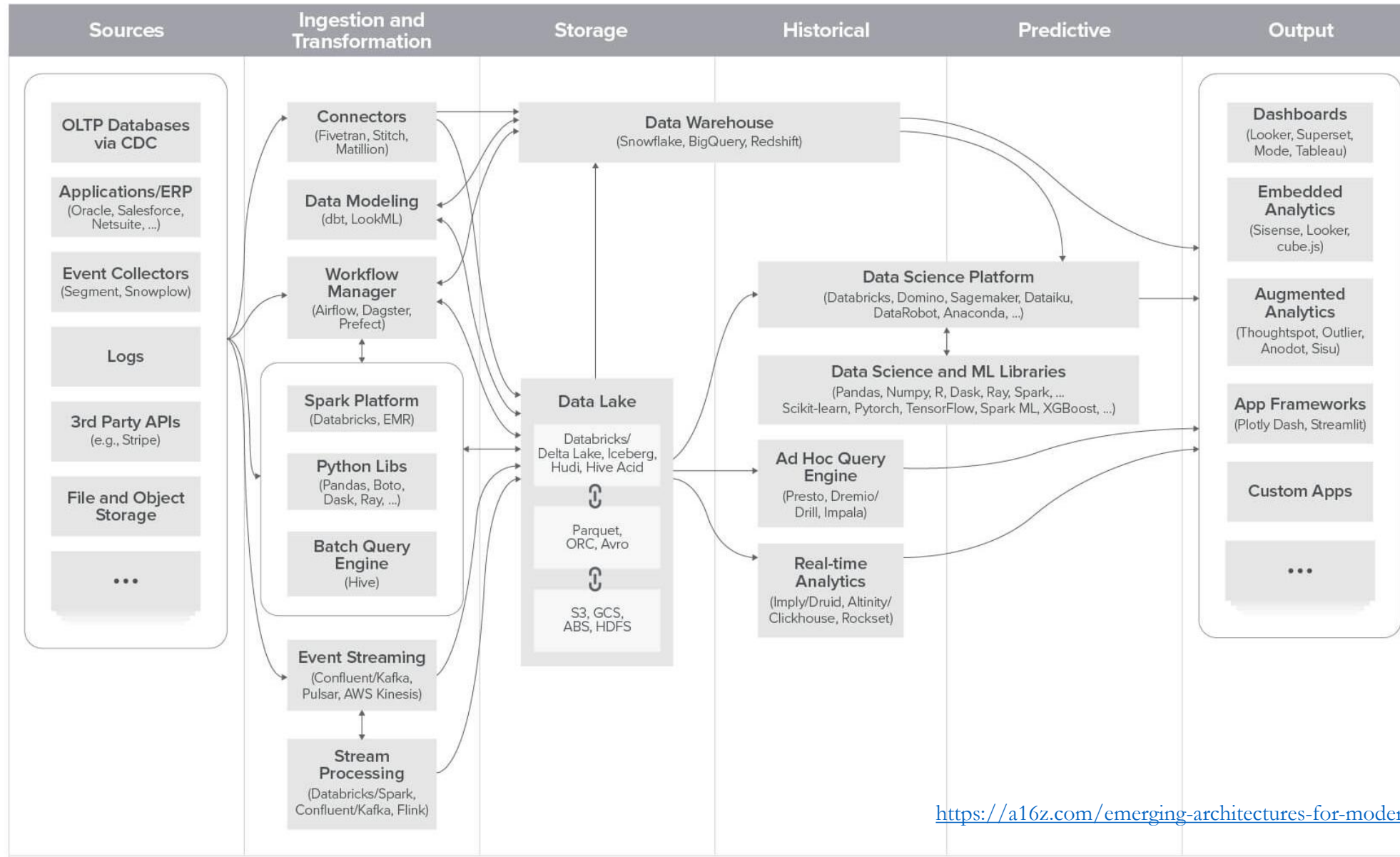
How long does it take to onboard developers in your organization?



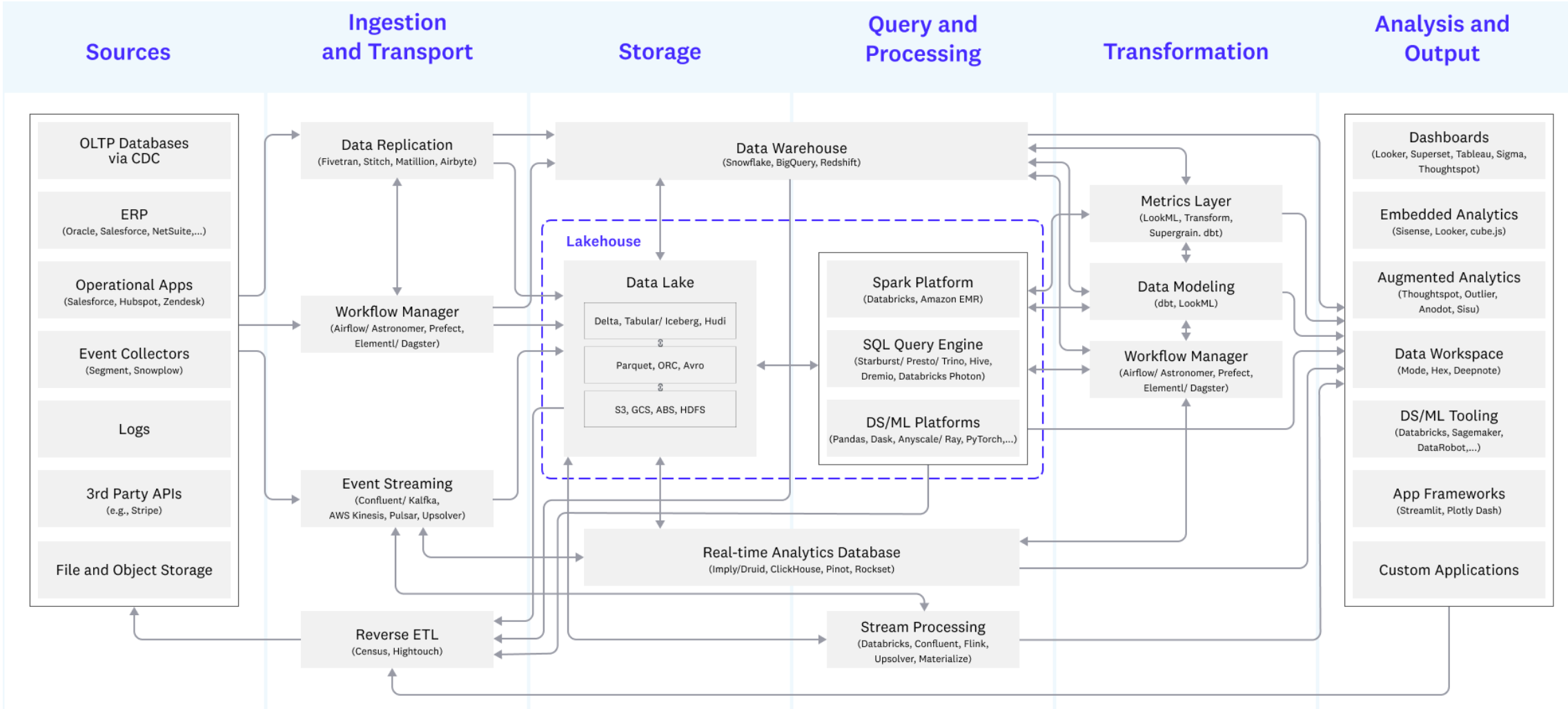
Analysis

- This year, GitLab's survey observed both AI usage and a DevSecOps platform to have a significant positive effect on developer onboarding.
- Respondents who said they are currently using AI for software development (43%) were much more likely than those not using AI (20%) to say that developer onboarding typically takes less than a month.
- The same effect was observed for DevSecOps platform usage, with 44% of those currently using a platform saying that developer onboarding takes less than a month, compared to 20% of those not using a platform.

Multimodal Data Processing: 2020



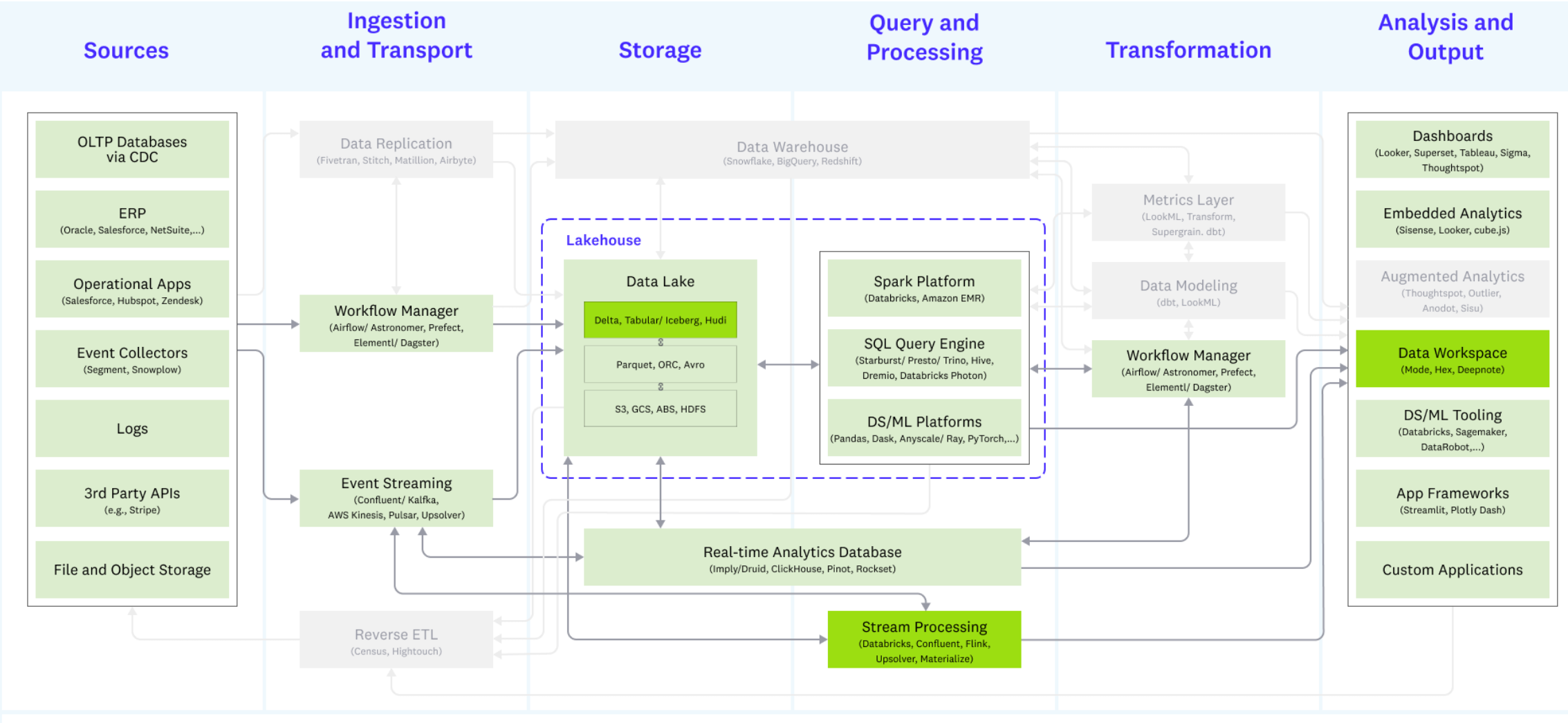
Multimodal Data Processing: 2024



Note: Darker boxes are new or meaningfully changed since v1 of the architecture in 2020; lighter colored boxes have remained largely the same. Gray boxes are considered less relevant to this blueprint.

<https://a16z.com/emerging-architectures-for-modern-data-infrastructure/>

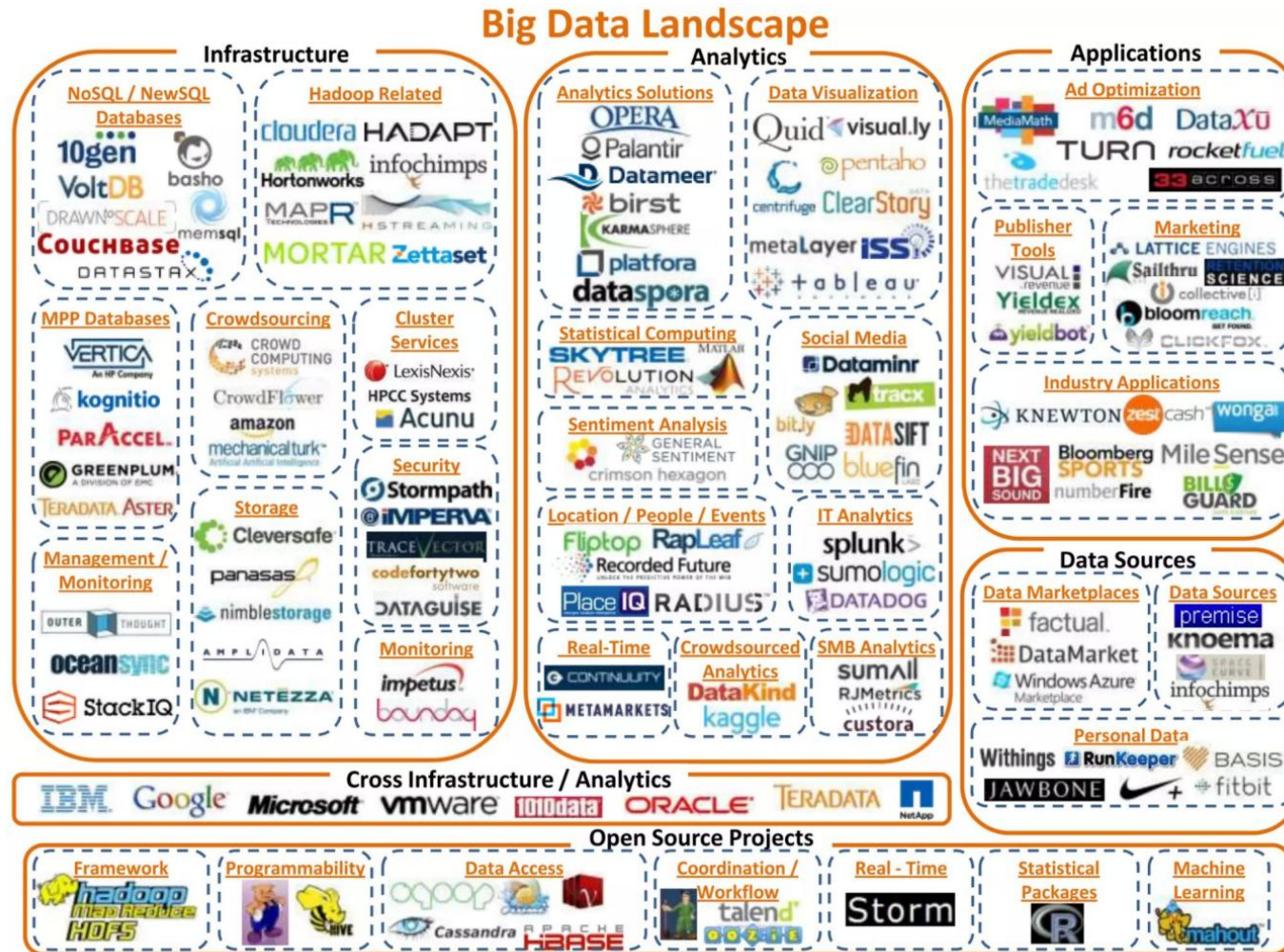
Multimodal Data Processing: 2024



Note: Darker boxes are new or meaningfully changed since v1 of the architecture in 2020; lighter colored boxes have remained largely the same. Gray boxes are considered less relevant to this blueprint.

<https://a16z.com/emerging-architectures-for-modern-data-infrastructure/>

Big Data Landscape 2012: Many tools, master one each, don't try to learn all!



© Matt Turck (@mattturck) and ShivonZilis (@shivonz)

<https://landscape.cncf.io/>

Big Data Landscape 2024: Even more tools...

The image displays a vast landscape of cloud-native tools, organized into five main functional areas, each with multiple sub-categories. The tools are represented by their respective logos, many of which include a 'CNCF' (Cloud Native Computing Foundation) logo, indicating their status as CNCF projects.

- App Definition & Development:** Includes sub-categories like Application Definition & Image Build, Database, Continuous Integration & Delivery, and Streaming & Messaging.
- Orchestration & Management:** Includes sub-categories like Scheduling & Orchestration, Service Mesh, Remote Procedure Call, Service Proxy, API Gateway, and Coordination & Service Discovery.
- Runtime:** Includes sub-categories like Cloud Native Storage, Cloud Native Network, and Container Runtime.
- Provisioning:** Includes sub-categories like Security & Compliance, Automation & Configuration, Container Regi..., and Key Managem....
- Observability & Analysis:** Includes sub-categories like Observability, Continuous Optimization, Chaos Engi..., and Feature Flagging.

<https://landscape.cncf.io/>

Too many tools to learn: Focus on an a future-proof, sustainable tech stack that (i) allows you to build useful solutions and (ii) interact with others (harboring a different skill set)

- › **The "Analytical" Tech Stack:** Designed for deep data analysis and business intelligence, this stack includes Snowflake for cloud-based data warehousing, dbt for data transformation, Airflow for orchestrating workflows, Iceberg for table formats, and Tableau for data visualization.
- › **The "Low Latency" Tech Stack:** Optimized for real-time data processing and ultra-fast querying, this stack leverages Kafka for event streaming, Flink or Spark Streaming for data stream processing, Iceberg for table storage, Redis for caching, and Apache Pinot for high-performance analytics.
- › **The "Big Data" Tech Stack:** Designed for large-scale data processing and analytics, this stack incorporates Databricks for collaborative data engineering, Spark for big data computations, Delta Lake or Iceberg for data lakehouses storage, and Apache Superset for customizable data visualization.
- › **The "LLM" Tech Stack:** Engineered for leveraging Large Language Models, this stack includes Neo4j for graph databases, Pinecone for vector similarity search, Claude or ChatGPT for AI-driven text generation, and Predibase for operationalizing machine learning models.
- › **The "Azure" Tech Stack:** Aligned with Microsoft's cloud ecosystem, this stack utilizes Microsoft Fabric for unified data management, Azure Data Factory for ETL workflows, Azure Synapse for analytics and big data integration, and PowerBI for enterprise-level data visualization.

Data Products

1



Sold over 100 M Alexa virtual assistants – estimated to cover at least 1 in 3 U.S. households

2



Generate more than 80% of content views through ML recommendation systems

3



Increased booking conversation by 4% by modelling likelihood of host acceptance

4



Save 100M driving miles and \$350-400M per year with ORION route optimization

5



Reduced methane emissions by 74% and increased production by 22% in wells with Kelvin sensor systems

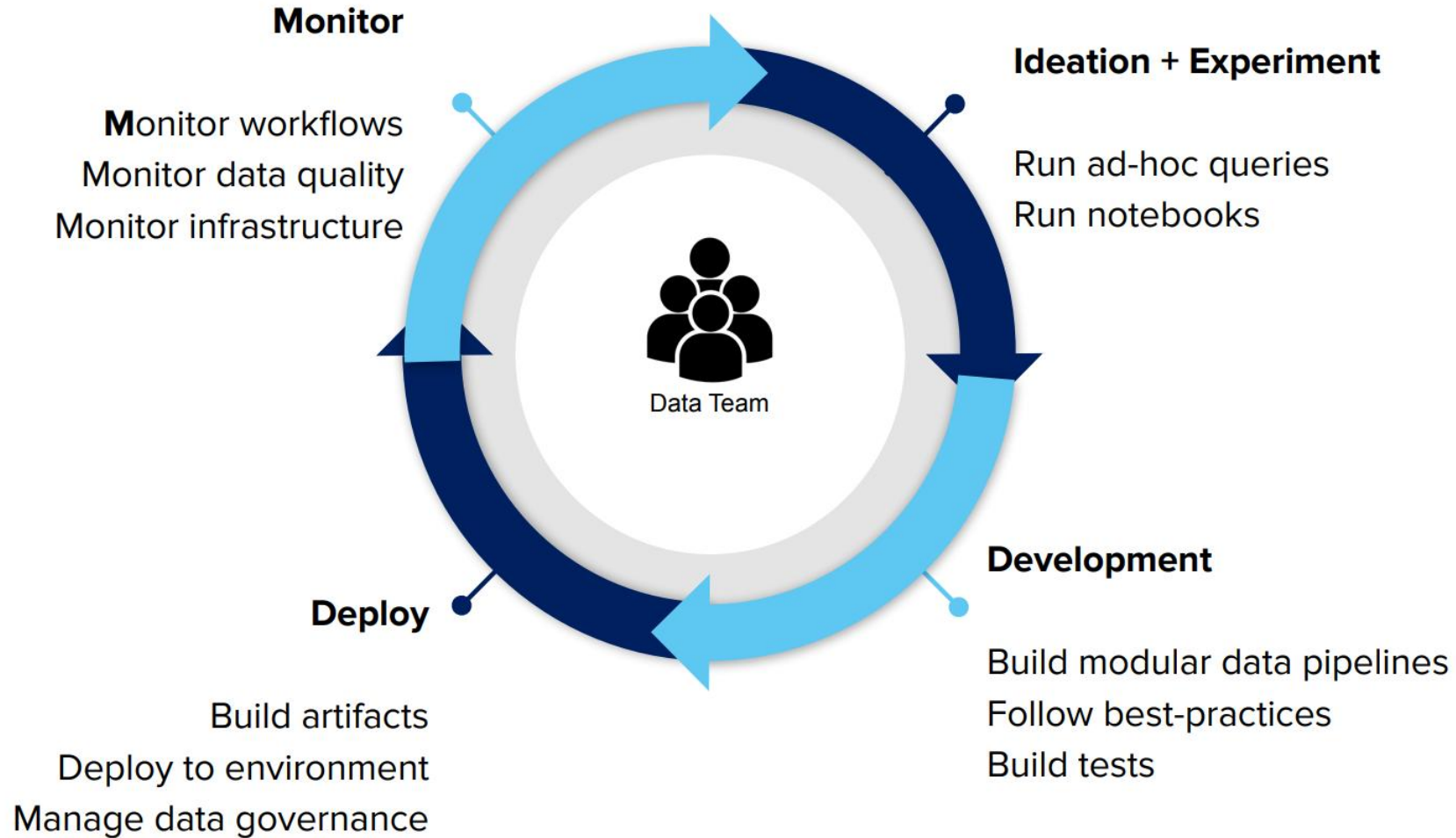
6



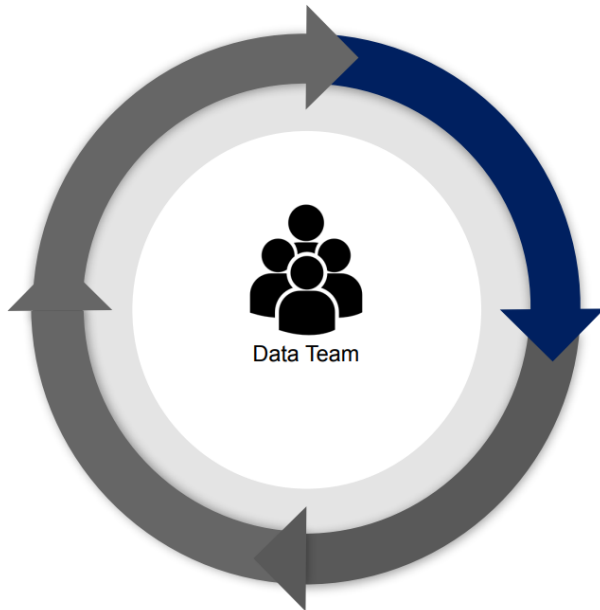
Earned three billion in revenue in 2019 selling aggregated data an analytics services.

<https://a16z.com/emerging-architectures-for-modern-data-infrastructure-2020/>

All data products go through the same 4 phases in their lifetime

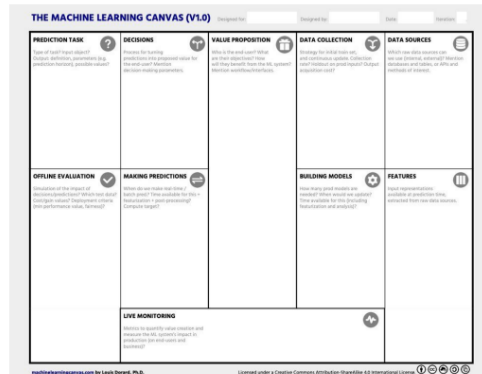


Ideation & Experimentation

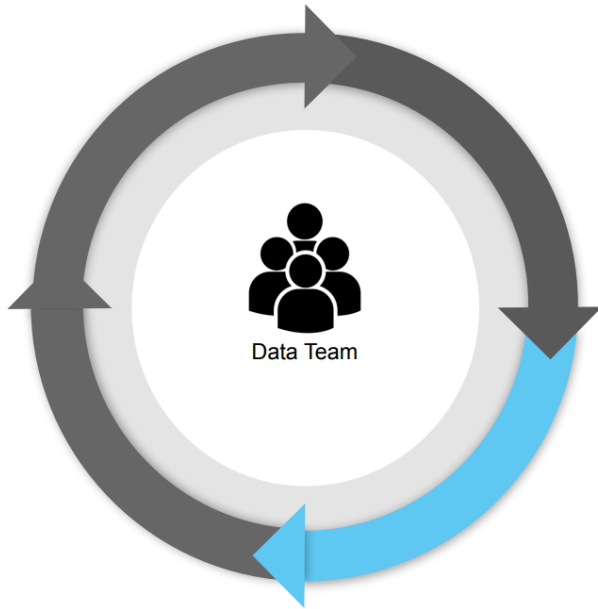


Deliverables:

- Business Opportunity is described
- Clear vision of what the actual data product is
- Completed Templates or kick off meeting
- Mock up of the actual product



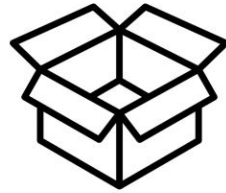
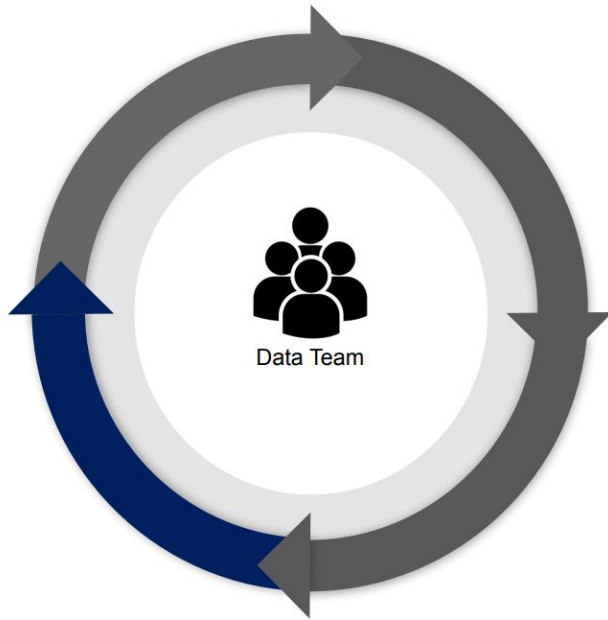
Development



Deliverables:

- Data & AI pipelines running green in: all envs
- Serving layer in place
- Documentation (model & pipelines) in place
- Guidelines and best practices

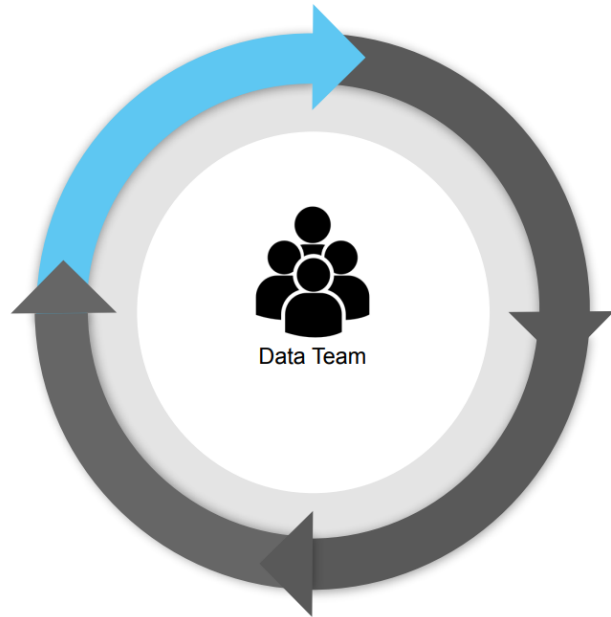
Industrialization



Deliverables:

- Alerting in place
- Data sources monitoring in place
- CI/CD pipeline in place
- Automation ready for day-to-day ops.
- training materials
- quality and governance plan
- Test plan and test cases

Monitor



Deliverables:

- Identify the current shortcomings of the solution
- Gather feedback
- Issue management process in place
- Data product usage statistics
- Data product evolution plan

How we go from a notebook PoC to a production grad application?

Model code

```
from diffusers import StableDiffusionPipeline, DPMSolverMultistepScheduler
import torch

model_id = "stabilityai/stable-diffusion-2-1"

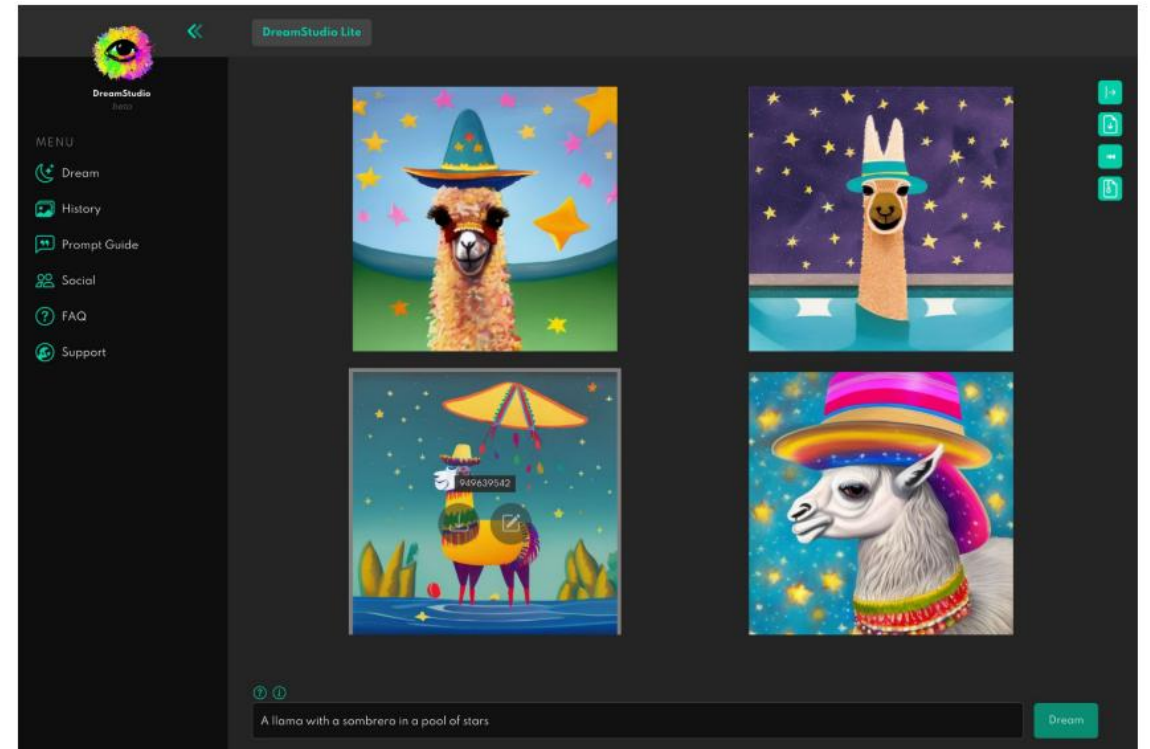
# Use the DPMSolverMultistepScheduler (DPM-Solver++) scheduler here instead
pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float16)
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
pipe = pipe.to("cuda")

prompt = "a picture of a camel in the desert without gravity"
image = pipe(prompt).images[0]

image.save("camel_zero_gravity.png")
```

Downloading: 100% 539/539 [00:00<00:00, 20.3kB/s]
Fetching 13 files: 77% 10/13 [00:35<00:08, 3.00s/it]
Downloading: 100% 342/342 [00:00<00:00, 16.7kB/s]
Downloading: 100% 345/345 [00:00<00:00, 18.2kB/s]
Downloading: 100% 633/633 [00:00<00:00, 37.0kB/s]
Downloading: 100% 1.36G/1.36G [00:17<00:00, 82.2MB/s]

Application



Creating a product is more than building a model

Perception of ML Workloads

Model code

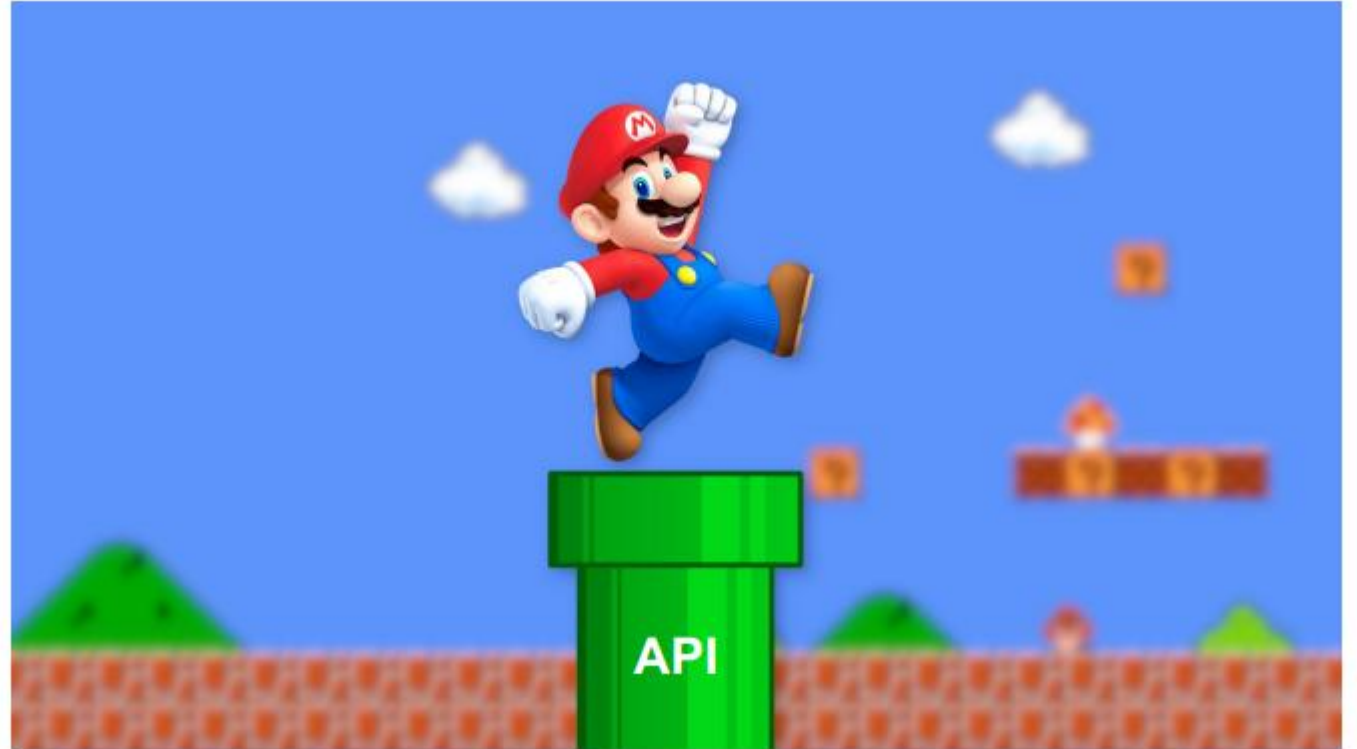
Reality of ML workloads

Data ingestion	Concept drift monitoring	Data drift monitoring
Feature storage	Retraining	Dependency upgrades
Model code	Resource scaling	Security
Model versioning	Pipeline monitoring	A/B testing traffic routing
Model API serving	Accuracy tracking	Maintenance
Data access	Data versioning	Lineage

Different components need to communicate with each other via agreed upon interfaces

Application Programming Interfaces (API)

- › It's the job of a data engineer / platform engineer / ML engineer to glue together these APIs into a coherent, robust whole
- › Data engineering = API plumbing
- › Data Engineer = Digital Plumber



Examples of paradigm-shifting APIs and API frameworks

1



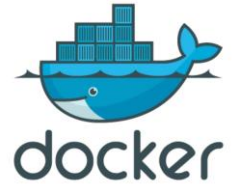
A distributed version control system that revolutionized how developers manage and collaborate on code.

2



A fast, in-memory data processing framework that transformed big data analytics with scalable, real-time computation.

3



A platform that popularized containerization, enabling consistent, portable environments for application deployment.

4



An infrastructure-as-code tool that simplifies provisioning and managing cloud resources across different providers.

5



A distributed streaming platform that excels in real-time data processing and event-driven architectures.

6



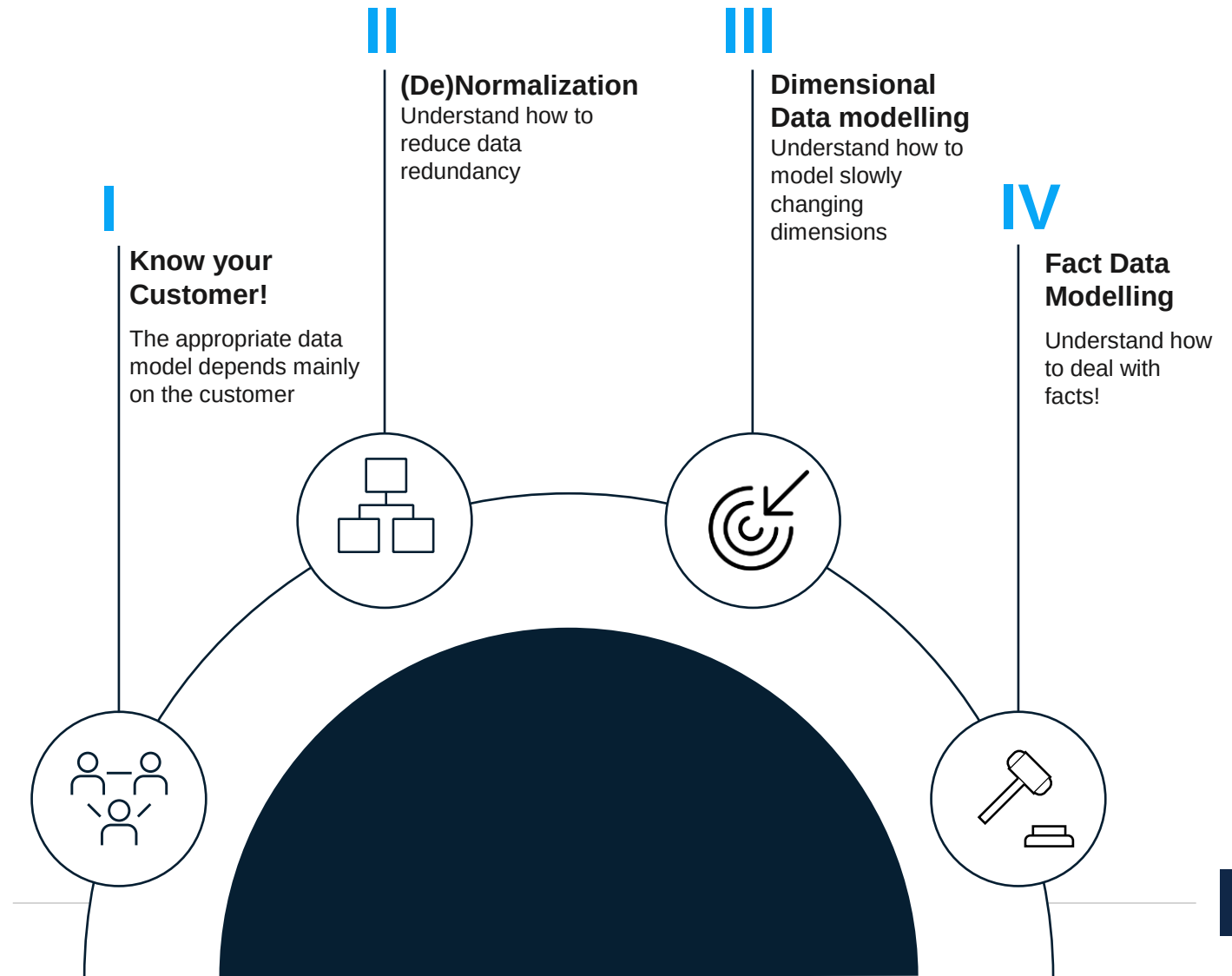
An orchestration system that automates deployment, scaling, and management of containerized applications.

Data Modelling

Data modelling is the process of designing structured representations of data to ensure efficiency, consistency, and scalability in data storage, management, and analysis.

Data Modelling is the key prerequisite for successful DE projects

In essence, data modelling provides the structure and logic for working with data effectively. Without it, systems become inefficient, hard to manage, and prone to errors. In data engineering, where data flows through complex pipelines and resides in various storage systems, a solid data model ensures the entire ecosystem remains performant, reliable, and aligned with business needs.



Data modelling is typically divided into three stages: conceptual, logical, and physical.

Each stage provides a progressively detailed view of the data, focusing on high-level relationships, detailed data structures, and implementation specifics, respectively.

Conceptual Data Model

- › Represents a high-level, abstract view of the data and its relationships.
- › Focused on defining entities and their relationships without technical details.
- › Used primarily for aligning business requirements with technical design.
- › Example: "A customer places an order for products."

Logical Data Model

- › Expands the conceptual model with detailed data structures like attributes and relationships.
- › Defines data types, keys (primary and foreign), and normalization to avoid redundancy.
- › Independent of any specific database or storage technology.
- › Focuses on ensuring data integrity, consistency, and clarity.
- › Example: A "Customer" entity has attributes like Customer_ID and Name

Physical Data Model

- › Implements the logical model in a specific database or storage system.
- › Includes technical details like indexing, partitioning, sharding, and constraints.
- › Optimized for performance, storage, and scalability.
- › Considers the limitations and capabilities of the chosen database or platform.
- › Example: A "Customer" table with columns defined as Customer_ID (INT), Name (VARCHAR)

Modelling the data appropriately to your use case and target consumers is key to unlock business value — know your customer!

- › OLTP vs OLAP:

- › **O**nline **T**ransactions **P**rocessing (OLTP): "I want to update one record" (`where customer_id=1203`)

- › **O**nline **A**nalytical **P**rocessing (OLAP): "I want to analyse many records"

- › Consumer groups:

- › Non computer people: Graphs,

- › Data Analysts: Simple tables (not complex data types, only STRING, FLOAT, INT, DATETIME), something that you can query with WHERE and GROUPBY

- › Data Engineers: complex types (STRUCT, ARRAY,...)

- › Applications (and their software engineers): typically, OLTP systems

OLAP is designed for analytical queries and decision-making, while OLTP focuses on real-time transaction processing and maintaining data integrity.

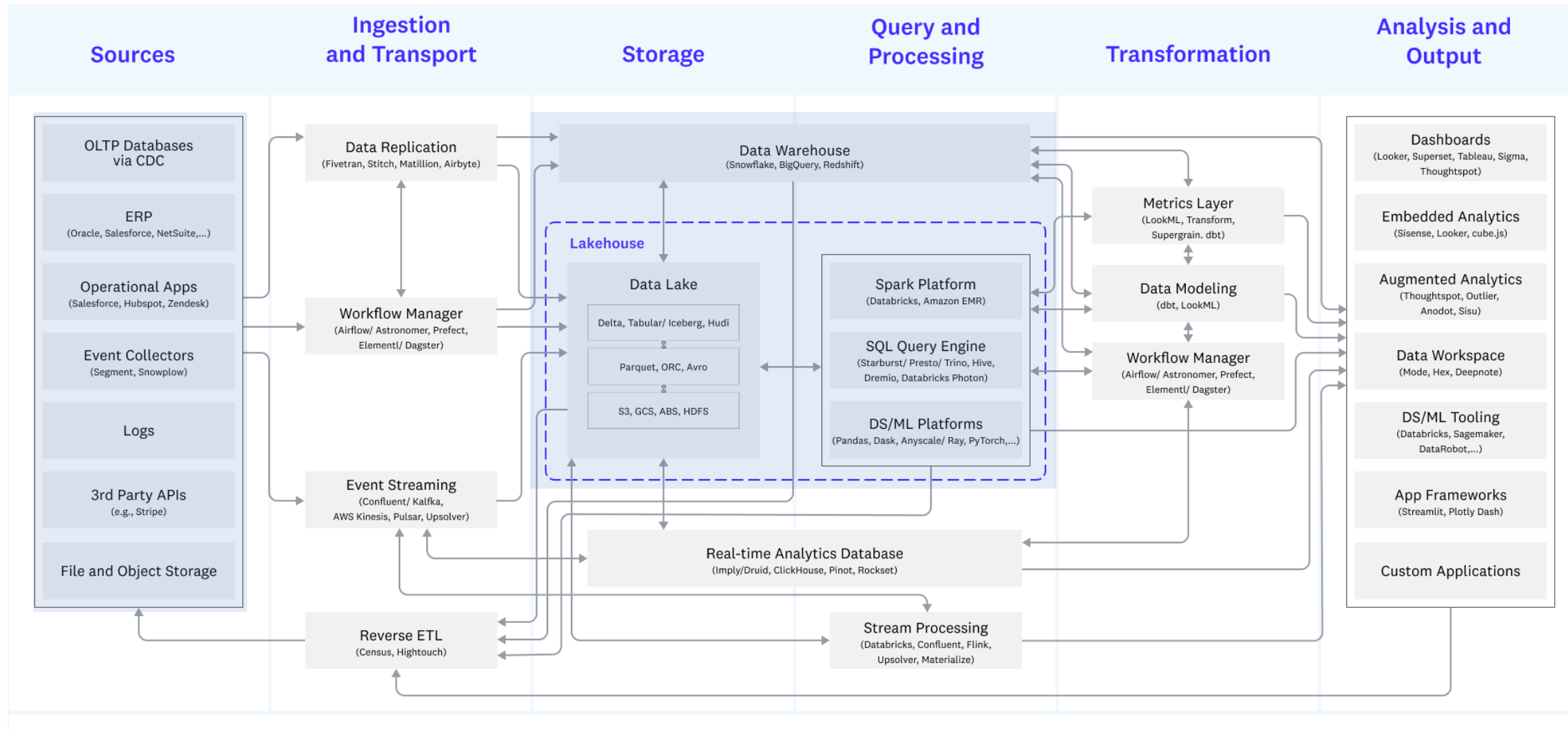
OLTP (Online Transaction Processing)

- › Used for **real-time transaction processing** (e.g., banking, e-commerce).
- › Optimized for **write-heavy, frequent updates, inserts, and deletes**.
- › Stores **current, operational data** in highly normalized tables.
- › Prioritizes **data integrity** and consistency through normalization (3NF or higher).
- › Designed for **minimal latency** and handling concurrent transactions.
- › Examples: Customer order systems, inventory management.

OLAP (Online Analytical Processing)

- › Used for **analysis, reporting, and decision support**.
- › Optimized for **read-heavy, complex queries** involving large datasets.
- › Stores **historical data** in a data warehouse or data mart.
- › Often uses **star or snowflake schemas** for efficient querying.
- › Prioritizes **query performance** over data redundancy or write efficiency.
- › Examples: Business intelligence dashboards, sales analysis.

OLTP is at the source of data collection, OLAP is down the road; as Data Engineers we tend to care more about OLAP schemas. Two popular ones are Star and Snowflake



Note: Darker boxes are new or meaningfully changed since v1 of the architecture in 2020; lighter colored boxes have remained largely the same. Gray boxes are considered less relevant to this blueprint.

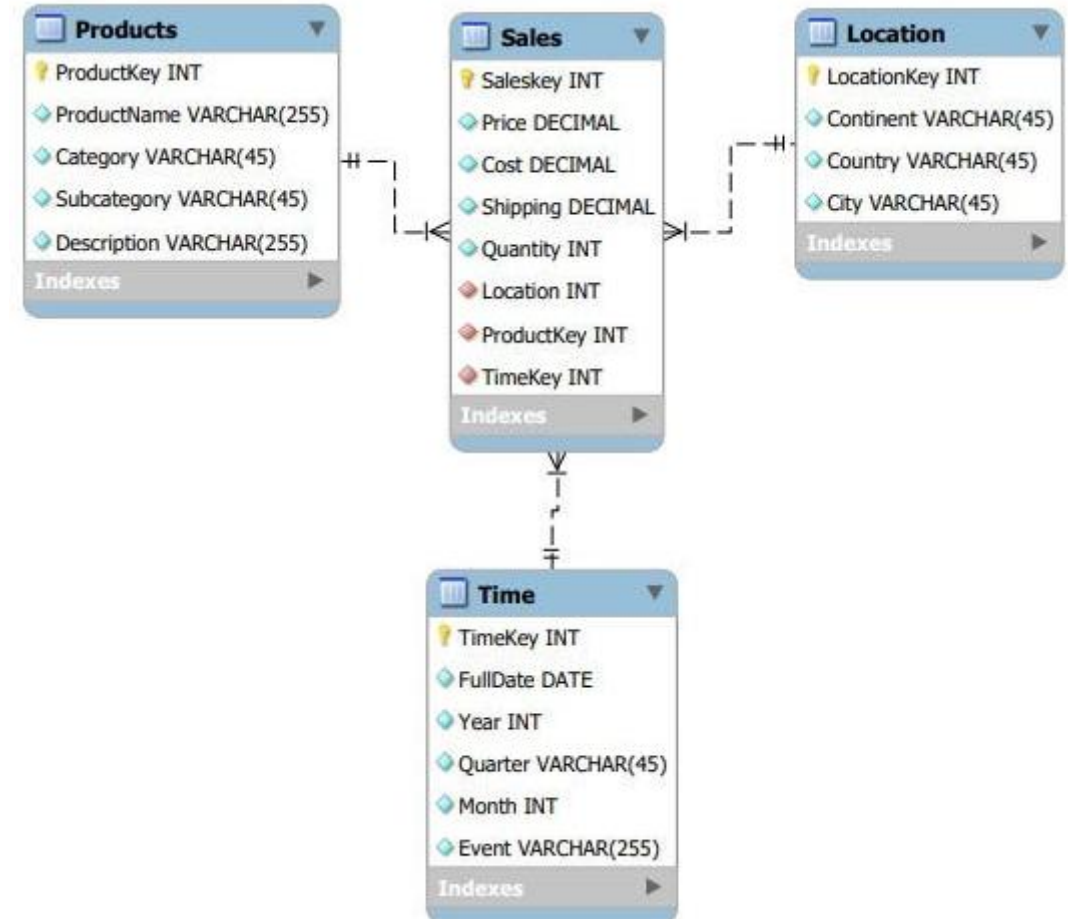
<https://a16z.com/emerging-architectures-for-modern-data-infrastructure/>

A star schema is a simple data model optimized for fast query performance in OLAP systems, with a single fact table and directly connected dimension tables.

Key considerations

- › Central **fact table** surrounded by directly **linked dimension tables**
- › **Denormalized** structure increases query speed.
- › Easy to design, understand, and query.
- › **Redundant data** in dimension tables.
- › Best suited for **small-to-medium-sized data marts**.
- › Optimized for **simple, high-level analytical queries**.

Example of a STAR schema



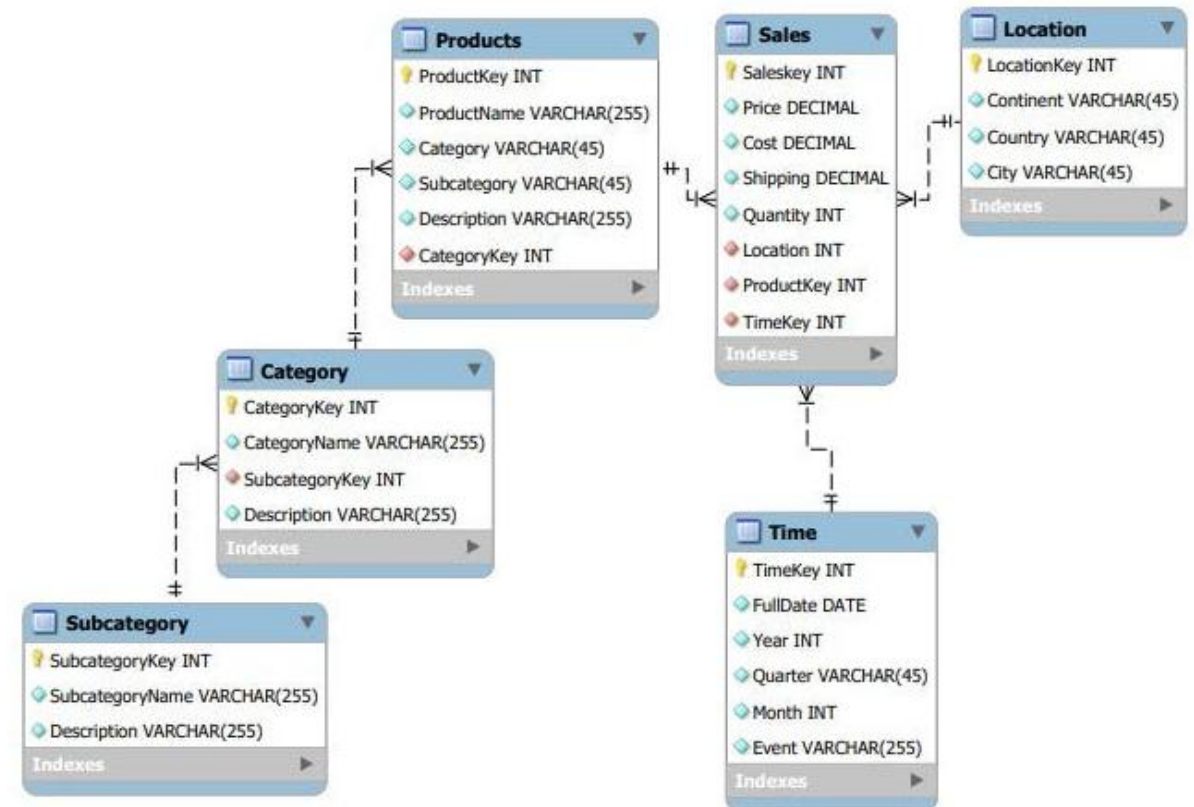
Source: <https://medium.com/art-of-data-engineering/data-modeling-understanding-the-star-and-snowflake-schemas-f786704977f6>

A snowflake schema is a more normalized, complex data model designed for efficient storage and scalability in OLAP systems, with dimension tables further split into sub-tables.

Key considerations

- › Central **fact table** with **normalized dimension tables**.
- › Reduces **data redundancy** at the cost of query performance.
- › Requires **more joins**, making queries slower and more complex.
- › Easier to scale for **large, complex data warehouses**.
- › Supports **multi-level hierarchical** data (e.g., product categories, locations).
- › Best suited for scenarios **prioritizing storage efficiency** and **data integrity**.

Example of a snowflake schema



Source: <https://medium.com/art-of-data-engineering/data-modeling-understanding-the-star-and-snowflake-schemas-f786704977f6>

Unnormalized Table: The Courses and Grades columns contain multiple values in a single cell (non-atomic values). Data redundancy and potential anomalies exist.

First Normal Form (1NF): Ensure that each column contains atomic values; eliminate repeating groups. In this example, course and grades contain multiple values in each cell. Break those up into multiple rows to arrive at 1NF.

StudentID	StudentName	Courses	Grades
1	Alice	Math, Physics	A, B
2	Bob	Math, Chemistry	B, A



StudentID	StudentName	Courses	Grades
1	Alice	Math	A
1	Alice	Physics	B
2	Bob	Math	B
2	Bob	Chemistry	A

Second Normal Form (2NF): Remove partial dependencies; ensure that every non-key attribute is fully functionally dependent on the entire primary key.

In our example, the full key (StudentID, Course) is not required to identify a student. This partial dependency needs to be removed by creating a separate table for students.

- › Identify Primary Key:
 - › Composite key: (StudentID, Course)
- › Functional Dependencies:
 - › StudentID $\rightarrow$ StudentName (Partial dependency)
 - › (StudentID, Course) $\rightarrow$ Grade
- › Issues:
 - › StudentName depends only on StudentID, not on the entire composite key. This violates 2NF due to the partial dependency.
- › Transformation: Create two separate tables to eliminate partial dependencies.
 - › The Students table captures data unique to each student.
 - › The StudentCourses table captures the relationship between students and courses, with Grade dependent on the full primary key (StudentID, Course).

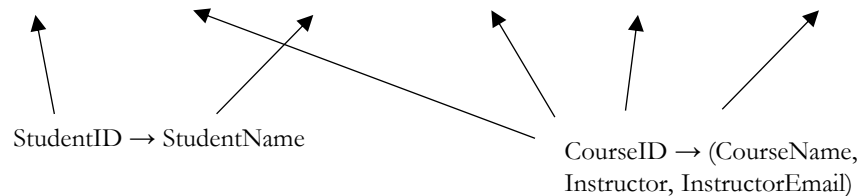
StudentID	StudentName
1	Alice
2	Bob

StudentID	Courses	Grades
1	Math	A
1	Physics	B
2	Math	B
2	Chemistry	A

In a more complicated case, we can see how we go from 1NF to 3NF. First, from 1NF to 2NF by removing partial dependencies

Now we have added instructor names and email as well as a course ID. (StudentID, CourseID) form a composite key. Several partial dependencies emerge which can be moved to separate tables.

Student ID	Course ID	Student Name	Courses	Instructor	InstructorEmail	Grades
1	101	Alice	Math	Prof. Smith	<u>smith@univ.edu</u>	A
1	102	Alice	Physics	Prof. Smith	<u>smith@univ.edu</u>	B
2	101	Bob	Math	Prof. Smith	<u>smith@univ.edu</u>	B
2	103	Bob	Chemistry	Prof. Brown	<u>brown@univ.edu</u>	A



In 2NF, all partial dependencies are eliminated, i.e. every attribute is now fully dependent on the primary key. However, Instructor → InstructorEmail is a transitive dependency

With three tables, only the course table contains transitive dependencies. The email address is bound to the instructor. To further reduce data duplication we can create an Instructor Table.

Student ID	Course ID	Student Name	Courses	Instructor	InstructorEmail	Grades
1	101	Alice	Math	Prof. Smith	smith@univ.edu	A
1	102	Alice	Physics	Prof. Smith	smith@univ.edu	B
2	101	Bob	Math	Prof. Smith	smith@univ.edu	B
2	103	Bob	Chemistry	Prof. Brown	brown@univ.edu	A

StudentID	StudentName
1	Alice
2	Bob

CourseID	CourseName	Instructor	InstructorEmail
101	Math	Prof. Smith	smith@univ.edu
102	Physics	Prof. Smith	smith@univ.edu
103	Chemistry	Prof. Brown	brown@univ.edu

StudentID	CourseID	Grades
1	101	A
1	102	B
2	101	B
2	103	A

Instructor → InstructorEmail

In 3NF we eliminate transitive dependencies by ensuring all non-key attributes depend only on the primary key, not on other non-key attributes

We have now reach the 3NF and no more data redundancy is present.

Student ID	Course ID	Student Name	Courses	Instructor	InstructorEmail	Grades
1	101	Alice	Math	Prof. Smith	smith@univ.edu	A
1	102	Alice	Physics	Prof. Smith	smith@univ.edu	B
2	101	Bob	Math	Prof. Smith	smith@univ.edu	B
2	103	Bob	Chemistry	Prof. Brown	brown@univ.edu	A

StudentID	StudentName
1	Alice
2	Bob

CourseID	CourseName	Instructor
101	Math	Prof. Smith
102	Physics	Prof. Smith
103	Chemistry	Prof. Brown

Instructor	InstructorEmail
Prof. Smith	smith@univ.edu
Prof. Brown	brown@univ.edu

StudentID	CourseID	Grades
1	101	A
1	102	B
2	101	B
2	103	A

Normalization is ideal for ensuring data integrity and reducing redundancy, while denormalization trades redundancy for improved performance in specific use cases.

In proper normalization choice can lead to severe problems, which is why understanding the difference is crucial

When to Normalize

- › To eliminate data redundancy and improve data integrity.
- › When you need to minimize data anomalies (insertion, update, and deletion anomalies).
- › To ensure that each attribute depends only on the primary key (apply 1NF, 2NF, 3NF, etc.).
- › When data consistency is critical across multiple applications or users.
- › To simplify future database maintenance and schema evolution.

When to Denormalize

- › To optimize query performance in read-heavy systems, such as data warehouses.
- › When frequent joins across normalized tables are causing significant performance bottlenecks.
- › To reduce query complexity and improve reporting or analytical speed.
- › When data redundancy is acceptable for faster lookups.
- › In scenarios where denormalized views can reduce application-layer logic.

In data modelling, Slowly Changing Dimensions (SCD) refer to data attributes in a dimension table that change slowly, as opposed to rapidly changing transactional data.

These changes typically occur in descriptive attributes (e.g., customer name, address, product details) and present a challenge for maintaining historical accuracy while supporting current data analysis.

SCD Type	Description
0 Fixed	- No changes are allowed; historical data remains unchanged. - Example: A product ID or SKU that is immutable.
1 Overwrite	- Old data is overwritten with new data. - Use Case: When historical accuracy is not required. - Example: Updating a customer's address directly without retaining the old address.
2 Add row	- A new record (row) is added for each change, with a versioning mechanism. - Use Case: When both current and historical data need to be preserved. - Example: Tracking changes to a customer's address over time.
3 Add Column	- A new column is added to store the previous value. - Use Case: When only the current and one prior value are needed. - Example: Adding a "Previous Address" column to a customer table.
4 Separate History Table	- Historical changes are stored in a separate table. - Use Case: When managing historical data outside the main dimension table. - Example: Moving all historical addresses to a CustomerHistory table.
6 Hybrid	- Combines Type 1, Type 2, and sometimes Type 3 approaches to maintain flexibility. - Use Case: When current, historical, and partial prior values are all required.

In data modelling, Slowly Changing Dimensions (SCD) refer to data attributes in a dimension table that change slowly, as opposed to rapidly changing transactional data.

These changes typically occur in descriptive attributes (e.g., customer name, address, product details) and present a challenge for maintaining historical accuracy while supporting current data analysis.

Type	Example	Pros & Cons																								
SCD 0	<table><tr><th>CID</th><th>Name</th><th>Address</th></tr><tr><td>1</td><td>Alice</td><td>123 Elm Street</td></tr><tr><td>2</td><td>Bob</td><td>456 Oak Ave</td></tr></table>	CID	Name	Address	1	Alice	123 Elm Street	2	Bob	456 Oak Ave	No changes possible in this type!															
	CID	Name	Address																							
	1	Alice	123 Elm Street																							
	2	Bob	456 Oak Ave																							
SCD 1 (Overwrite)	<table><tr><th>CID</th><th>Name</th><th>Address</th></tr><tr><td>1</td><td>Alice</td><td>789 Sun Blvd</td></tr><tr><td>2</td><td>Bob</td><td>456 Oak Ave</td></tr></table>	CID	Name	Address	1	Alice	789 Sun Blvd	2	Bob	456 Oak Ave	- Simple, minimal storage required - No historical data available															
	CID	Name	Address																							
	1	Alice	789 Sun Blvd																							
	2	Bob	456 Oak Ave																							
SCD 2 (Add row)	<table><tr><th>CID</th><th>Name</th><th>Address</th><th>StartDate</th><th>EndDate</th><th>IsCurrent</th></tr><tr><td>1</td><td>Alice</td><td>123 Elm Street</td><td>2024-01-01</td><td>2024-06-01</td><td>No</td></tr><tr><td>1</td><td>Alice</td><td>789 Sun Blvd</td><td>2024-06-01</td><td>NULL</td><td>Yes</td></tr><tr><td>2</td><td>Bob</td><td>456 Oak Ave</td><td>2024-01-01</td><td>NULL</td><td>Yes</td></tr></table>	CID	Name	Address	StartDate	EndDate	IsCurrent	1	Alice	123 Elm Street	2024-01-01	2024-06-01	No	1	Alice	789 Sun Blvd	2024-06-01	NULL	Yes	2	Bob	456 Oak Ave	2024-01-01	NULL	Yes	- Retains full historical data - Increases table size, requires careful query handling to filter current records
	CID	Name	Address	StartDate	EndDate	IsCurrent																				
	1	Alice	123 Elm Street	2024-01-01	2024-06-01	No																				
	1	Alice	789 Sun Blvd	2024-06-01	NULL	Yes																				
2	Bob	456 Oak Ave	2024-01-01	NULL	Yes																					

In data modelling, Slowly Changing Dimensions (SCD) refer to data attributes in a dimension table that change slowly, as opposed to rapidly changing transactional data.

These changes typically occur in descriptive attributes (e.g., customer name, address, product details) and present a challenge for maintaining historical accuracy while supporting current data analysis.

Type	Example	Pros & Cons																												
SCD 3 (Add column)	<table><tr><th>CID</th><th>Name</th><th>Address</th><th>PrevAddress</th></tr><tr><td>1</td><td>Alice</td><td>789 Sun Blvd</td><td>123 Elm Street</td></tr><tr><td>2</td><td>Bob</td><td>456 Oak Ave</td><td></td></tr></table>	CID	Name	Address	PrevAddress	1	Alice	789 Sun Blvd	123 Elm Street	2	Bob	456 Oak Ave		- Simple, retains one historical value - Limited tracking																
	CID	Name	Address	PrevAddress																										
	1	Alice	789 Sun Blvd	123 Elm Street																										
2	Bob	456 Oak Ave																												
SCD 4 (Separate History Table)	<table><tr><th>CID</th><th>Name</th><th>Address</th><th>vld_fr</th><th>Vld_to</th></tr><tr><td>1</td><td>Alice</td><td>123 Elm Street</td><td>2024-01-01</td><td>2024-06-01</td></tr></table>	CID	Name	Address	vld_fr	Vld_to	1	Alice	123 Elm Street	2024-01-01	2024-06-01	- Keep main table compact, - detailed history in separate table - Requires joins to retrieve historical data																		
	CID	Name	Address	vld_fr	Vld_to																									
	1	Alice	123 Elm Street	2024-01-01	2024-06-01																									
SCD 6 (Hybrid)	<table><tr><th>CI D</th><th>Nam e</th><th>Address</th><th>Prev Address</th><th>StartDate</th><th>EndDate</th><th>IsCurrent</th></tr><tr><td>1</td><td>Alice</td><td>123 Elm Street</td><td></td><td>2024-01-01</td><td>2024-06-01</td><td>No</td></tr><tr><td>1</td><td>Alice</td><td>789 Sun Blvd</td><td>123 Elm Street</td><td>2024-06-01</td><td>NULL</td><td>Yes</td></tr><tr><td>2</td><td>Bob</td><td>456 Oak Ave</td><td></td><td>2024-01-01</td><td>NULL</td><td>Yes</td></tr></table>	CI D	Nam e	Address	Prev Address	StartDate	EndDate	IsCurrent	1	Alice	123 Elm Street		2024-01-01	2024-06-01	No	1	Alice	789 Sun Blvd	123 Elm Street	2024-06-01	NULL	Yes	2	Bob	456 Oak Ave		2024-01-01	NULL	Yes	- Retains full historical data - Increases table size, requires careful query handling to filter current records
	CI D	Nam e	Address	Prev Address	StartDate	EndDate	IsCurrent																							
	1	Alice	123 Elm Street		2024-01-01	2024-06-01	No																							
	1	Alice	789 Sun Blvd	123 Elm Street	2024-06-01	NULL	Yes																							
2	Bob	456 Oak Ave		2024-01-01	NULL	Yes																								

Fact data modelling is a data modelling technique primarily used in data warehousing and business intelligence systems to design databases that support analytical queries efficiently.

It focuses on organizing and structuring data to make it easier to retrieve meaningful insights. At its core, fact data modelling is about identifying facts and dimensions and organizing them into a schema, typically a star schema or snowflake schema.



Facts are things that happened

Think of facts as events that took place. To whom, where when did what happen?



Fact tables are much larger than dim tables

Dimensions tables tend to be small in comparison to facts. Think of user dims (name, address) and a fact table of their click stream



New facts are added often

Unlike (slowly changing) dimensions, fact values are continuously being added. Your data model should account for that

Fact or Dimension? As a rule of thumb, dimensions are in the **GROUP BY** statement, facts are in the aggregation function (**sum()**, **mean()**, **count()**) — but it is messy!

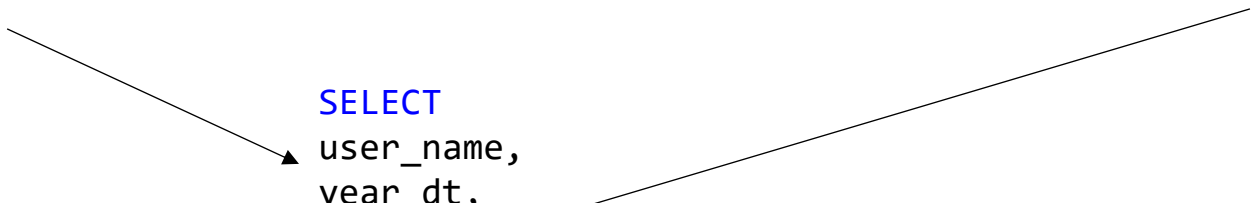
Sometimes it does depend on the context on what you want to model. Keep the user in mind. The line is blurry!

Dimensions

- › Dimensions are usually obtained from some snapshot of a state of the database.
- › Dimensions are often categories (male/female) or quasi categories (height in cm) and have “agnostic interpretation” (1.90m is tall, but 500EUR may be a small or big purchase)
- › Dimensions are typically used in **GROUP BY** statements when performing analytics

Facts

- › Facts occur, they don't change/evolve
 - › I made a purchase
 - › Even if I return the item, I don't “unpurchase” it
- › Facts have typically a point in time at which they “occurred”
- › Facts are typically used in the aggregation functions



```
SELECT
user_name,
year_dt,
sum(total_amount) as yearly_purchases
FROM orders
GROUP BY user_name, year_dt
```


Sampling, bucketing and retention periods can help to manage massive amounts of data

Sometimes it does depend on the context on what you want to model. Keep the user in mind. The line is blurry!

Dimensions

- › **Sampling**: for some ML/AI applications we can randomly sample big tables and get representative data for model training. Not valid for all cases though
- › **Retention periods**: Oftentimes we don't need all historic facts. Delete old ones or move them to a history table.
- › **Bucketing: Bucketing**: is a strategy where big fact tables and corresponding dimension tables are portioned/bucketed on the same key. This can drastically improve performance as it avoid costly shuffles.

Example using bucketing

```
s3://my-data/users/  
├── bucket_0/  
│   ├── part-00000.parquet  
│   └── part-00001.parquet  
├── bucket_1/  
│   └── part-00000.parquet  
├── bucket_2/  
│   └── part-00000.parquet  
└── bucket_3/  
    └── part-00000.parquet
```

```
s3://my-data/transactions/  
├── bucket_0/  
│   └── part-00000.parquet  
├── bucket_1/  
│   ├── part-00000.parquet  
│   └── part-00001.parquet
```

In a bucket join, data of the same bucket is loaded by the same worker, eliminating the need for costly data shuffle. Of course, this only works if tables are partitioned properly.

Most of the functionality is provided by Spark/Hive/Trino under the hood without the need for user intervention. But partition your tables properly!

Data is processed by the same worker: No shuffle!

- › Each bucket is processed by one worker:
 - › Bucket 0 → Worker 1
 - › Bucket 1 → Worker 2
 - › Bucket 2 → Worker 3
 - › Bucket 3 → Worker 4
- › All data of one bucket is on the same worker:
 - › `users/bucket_0` is joined with `transactions/bucket_0`.
 - › `users/bucket_1` is joined with `transactions/bucket_1`.

```
spark.sql("""
    SELECT u.user_id, u.name, t.amount
    FROM users u
    JOIN transactions t ON u.user_id = t.user_id
    """)
```

Hive/Spark/... Will automatically perform bucket joins

- › **Logical Plan:** Spark analyzes the query and detects that users and transactions are bucketed by `user_id` with the same bucket count. It decides to **use a bucket join** (a.k.a. bucketed sort-merge join).
- › **Physical Plan:** Spark assigns **a task for each bucket** (e.g., one task per bucket pair: `bucket_0_users` + `bucket_0_transactions`). Tasks are distributed across executors (worker nodes).
- › **Data Loading:** Executors read only the bucket files assigned to them (e.g., `bucket_0/` for both users and transactions).
- › **Join Execution:** Each executor joins its bucket pair independently. Results are combined into the final output without requiring shuffle.

A Datelist Table is a class of datasets, like a fact table, dimension table, or a cube

A Datelist table contains a cumulative list of users or some ids (one row per user/id who has ever been active), and for each user the Datelist stores a first active date, a last active date, and a history of past active dates

userId	activeDt
1	2022-01-01
2	2022-01-02
1	2022-01-03
2	2022-01-04
1	2022-01-05

Collapse the table into
one row per user

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	[2022-01-01, 2022-01-03, 2022-01-05]	2022-01-05
2	2022-01-01	2022-01-04	[2022-01-02, 2022-01-04]	2022-01-05

This is a complex data
type (`struct[varchar]`)
which will be very slow to
process and expansive to
store

A DatelistInt is a data structure that encodes multiple days of user activity in a single integer value (usually a BIGINT).

Each bit in the integer represents a calendar date for a rolling history, with the smallest bit representing the current date. If the user was active on a particular date, that bit is set to 1, otherwise it is set to 0.

› Current Date: 2022-01-05. User 1 was active on:

- › 2022-01-05 (0 days ago)
- › 2022-01-03 (2 day ago)
- › 2022-01-01 (4 days ago)

› Bit Representation: Each day is mapped to a bit:

- › 2022-01-05 → Bit 0 → 1 → $1 \cdot 2^0 = 1$
- › 2022-01-04 → Bit 1 → 0 → $0 \cdot 2^1 = 0$
- › 2022-01-03 → Bit 2 → 1 → $1 \cdot 2^2 = 4$
- › 2022-01-02 → Bit 3 → 0 → $0 \cdot 2^3 = 0$
- › 2022-01-01 → Bit 4 → 1 → $1 \cdot 2^4 = 16$
- › $\text{datelist_int} = 1 + 0 + 4 + 0 + 16 = 21 \equiv 10101$ (in bits)
- › Can store 63 days history (for 64bit integer BIGINT)

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	[2022-01-01, 2022-01-03, 2022-01-05]	2022-01-05
2	2022-01-01	2022-01-04	[2022-01-02, 2022-01-04]	2022-01-05

userId	firstActive	LastActive	dateListInt	date
1	2022-01-01	2022-01-05	21	2022-01-05
2	2022-01-01	2022-01-04	?	2022-01-05

The datelist int representation is much smaller!

Solve this in the exercise session

Once we have our user activity encoded in a Datelist Int, we can perform several useful transformations with extremely efficient bitwise operations without data shuffling!

Here are some examples of queries against a `datelist_int` using Presto's bitwise functions:

Example	Strategy	SQL demo code
1 Users active 5 days ago	Allows simple operations <code>CAST(POW(2, 5) AS INT) = 32 = 100000</code> (in bits) <code>BITWISE_AND(21, 32)</code>: 010101 (21 in binary), 100000 (32 in binary) have a 000000 “overlap” / “bitwise_and” (for <code>userId=1</code>) Hence, we find that user is not active 5 days ago using only simple mathematics, no string parsing etc.	<pre>SELECT userid FROM active_user_datelist WHERE BITWISE_AND(active_datelist_int, CAST(POW(2,5) AS INT)) > 0</pre> <code>BITWISE_AND(1010, 0110) = 0010</code>
2 Number of active days out of the last 7 days	8bit representation of $2^7 = 128 \equiv 1000000$. Subtracting one yields 0111111. Counting the matching days yields the result	<pre>SELECT userid, BIT_COUNT(BITWISE_AND(active_datelist_int, CAST(POW(2, 7) AS INT) - 1)) as 17 FROM active_user_datelist</pre>
3 Number of users who churned this week	Condition 1: The user was active at least once in the 8–14 days ago period. Conditions 2: The user had no activity in the most recent 7 days. Imagine doing this using group by!!	<pre>SELECT COUNT(1) AS churned_user_cnt FROM active_user_datelist WHERE BITWISE_AND(BITWISE_RIGHT_SHIFT(active_datelist_int, 7), CAST(POW(2,7) AS INT) - 1) > 0 AND BITWISE_AND(active_datelist_int, CAST (POW(2, 7) AS INT) - 1) = 0</pre>

Huge fact tables can severely slow down analysis if there are billions of records, e.g. lots of events of user activity

A fact table tracking individual user activity (E.g. likes, comments etc) gets really big, really fast. Same for click stream data or logs from IoT devices

Example of event log data

Granularity: userId, eventTime

userId	eventTime	action
1	2024-01-01T10:00:00	like
1	2024-01-01T10:00:00	comment
1	2024-01-03T10:00:00	comment
1	...	...
2	2024-01-01T10:00:00	comment
2	2024-01-01T10:00:00	like

Summary

- Each record is one event in a log stream
- User x at time t is liking something
- Analysing this data, merge with other data can be tricky given its size
- Do users from Germany like more or less compared to users in France?
- Merging country to userId from a user dimension table is a big, i.e. costly, join
- We can group by userId, action and day(eventTime) to get a smaller table

In aggregation tables we can summarize that information to have a more tractable object to work with

Merging this with dimensional data is already easier, since the amount of data to merge is reduced

Aggregated table (aggregated by user, date, action)

Granularity: userId, day-date, action

userId	date	metric_name	count
1	2024-01-01	likes_given	12
1	2024-01-01	comments_given	10
1	2024-01-02	likes_given	5
1	...		
2	2024-01-01	comments_given	6
...			

Summary

- Considerably smaller already
- We lost intra-day observability though

We got here by applying a query like:

```
SELECT userId, month(eventTime),  
action, count(1)  
FROM table
```

We can reduce the data even more by using arrays of data referencing a month

In the example we have monthly data for each user, and an array of counts for each metric. This comes at a cost though: We now need to merge SCDs at a monthly level, so if things change in a month, we have to sacrifice some precision for huge faster queries!

Long-array metrics to further collapse the table by a factor of 30

Granularity: userId, month_start, action

userId	month_start	metric_name	count_array
1	2024-01-01	likes_given	[3, 0 ,2 , ..., 2, 3, 15]
1	2024-02-01	comments_given	10
1	...		
2	2024-01-01	comments_given	6
...			
...			

Summary

- For each day, one value in the array
- This greatly reduces the size of the dataset

The “Justin Bieber problem” is a general issue (often tied to celebrities) that occurs if a small portion of the data receives disproportionately high traffic.

In this cases it is advisable to pre-calculate the required quantities, rather than computing them on the fly.

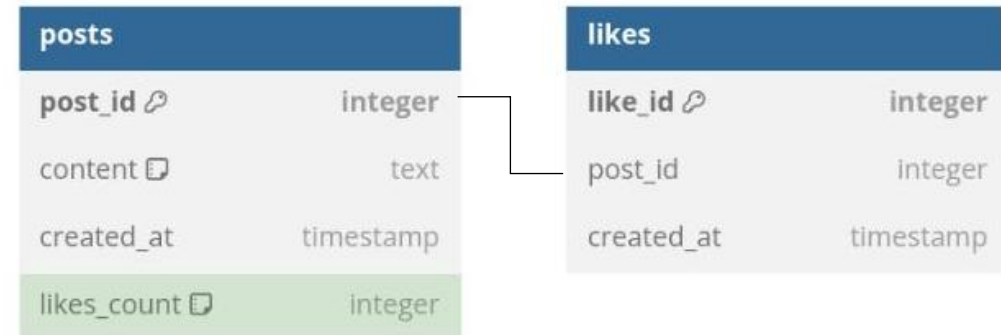
Data is processed by the same worker: No shuffle!

- › For example, in the case of likes count: each time a user requests a post, the post data, including the likes count, needs to be retrieved from the database. To calculate the likes count, we must aggregate the number of rows in the "likes" table for that post. This becomes very slow when millions of rows need to be aggregated each time the post data is requested
- › This is Where Aggregated Values Come Into Play.
Instead of recalculating the likes count on every request, the solution was to precompute the total number of likes and store it directly in the post table as a likes_count. This count is updated incrementally every time a like or unlike occurs, allowing the system to display the total instantly without querying and counting individual rows.

Source: moez zhioua on linkedin:

https://www.linkedin.com/posts/moez-zhioua_ever-heard-of-the-justin-bieber-problem-in-activity-7266724225112047617-CLLe?utm_source=share&utm_medium=member_desktop

The Justin Bieber Problem



```
SELECT COUNT(*)  
FROM likes  
WHERE post_id = <post_id>;
```



```
SELECT likes_count  
FROM posts  
WHERE post_id = <post_id>;
```

